DineDash Food Delivery Data Analysis

Azure Databricks Capstone Report

Prepared by
Amey Vishnu Mahajani
mahajaniameyvishnu@gmail.com

Project Background

The food delivery industry has experienced rapid growth over the last decade, driven by the increasing demand for convenience, improved delivery logistics, and the expansion of digital platforms. MealDash has revolutionized the way food is ordered and delivered, creating new opportunities for restaurant owners and consumers alike.

However, as the industry continues to expand, food delivery platforms face increasing competition, rising customer expectations for faster and more personalized services, and challenges in optimizing their operations across a diverse and extensive menu offering. The case study focuses on using big data and advanced analytics to streamline operations, optimize delivery times, enhance customer experience, and improve menu offerings.

Key Dataset Attributes

Customers: customer_id, name, email, signup_date.

Restaurants: restaurant_id, name, cuisine, location_id.

Menu Items: item_id, restaurant_id, item_name, price.

Orders: order_id, timestamp, customer_id, restaurant_id, agent_id, delivery_location_id, items_ordered, total_amount, tip, payment_method, order_status.

Delivery Agents: agent_id, name, phone, rating.

Proposed Solution Scope

The proposed architecture uses Databricks and Apache Spark with a data lake, ETL pipelines using Delta Live Tables/Lakeflow, streaming processing, and analytics dashboards. The executed work in this report focuses on ETL pipelines, streaming data analytics, dashboards, and workflow orchestration.

Business Needs

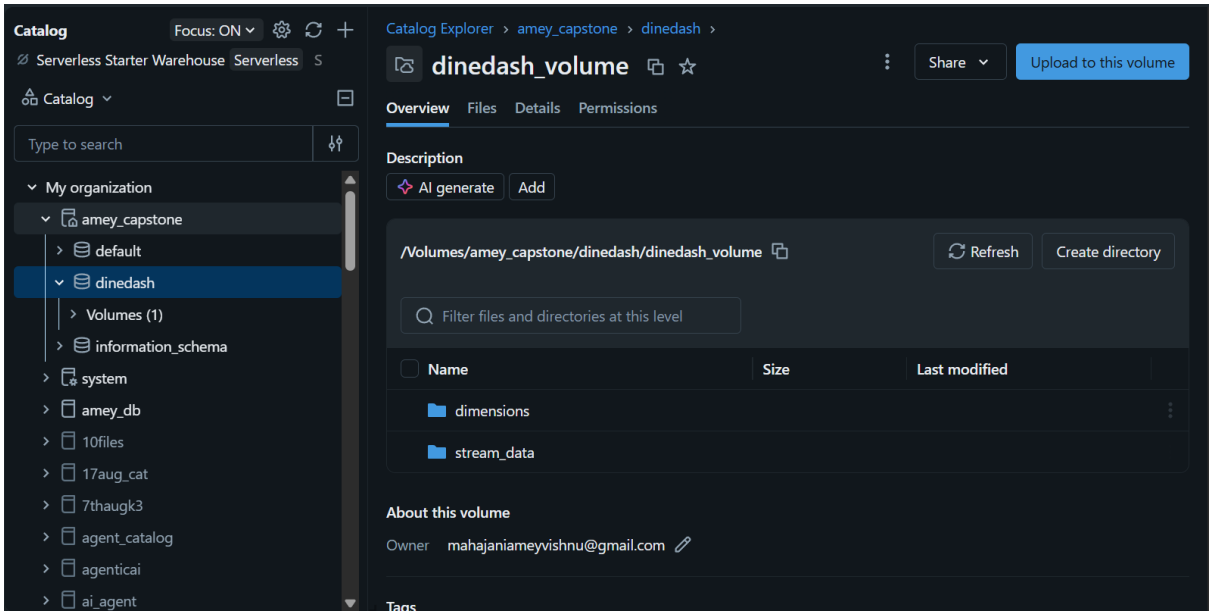
The case study identifies the need for data integration, real-time analytics, predictive analytics, optimization algorithms, personalization, and continuous KPI monitoring. This report documents the implemented data-engineering scope: ingestion, transformation, incremental processing, streaming analytics, data quality, dashboards, and workflow orchestration.

Key Challenges

1. Data Fragmentation: Different systems for orders, customer profiles, menus, and delivery logs lead to fragmented data, making it difficult to analyze information comprehensively.
2. Customer Segmentation: Understanding customer preferences from historical data is essential for personalized promotions, recommendations, and loyalty rewards.
3. Order Prediction: Predicting order volumes across regions and times helps optimize delivery capacity and reduce wait times.

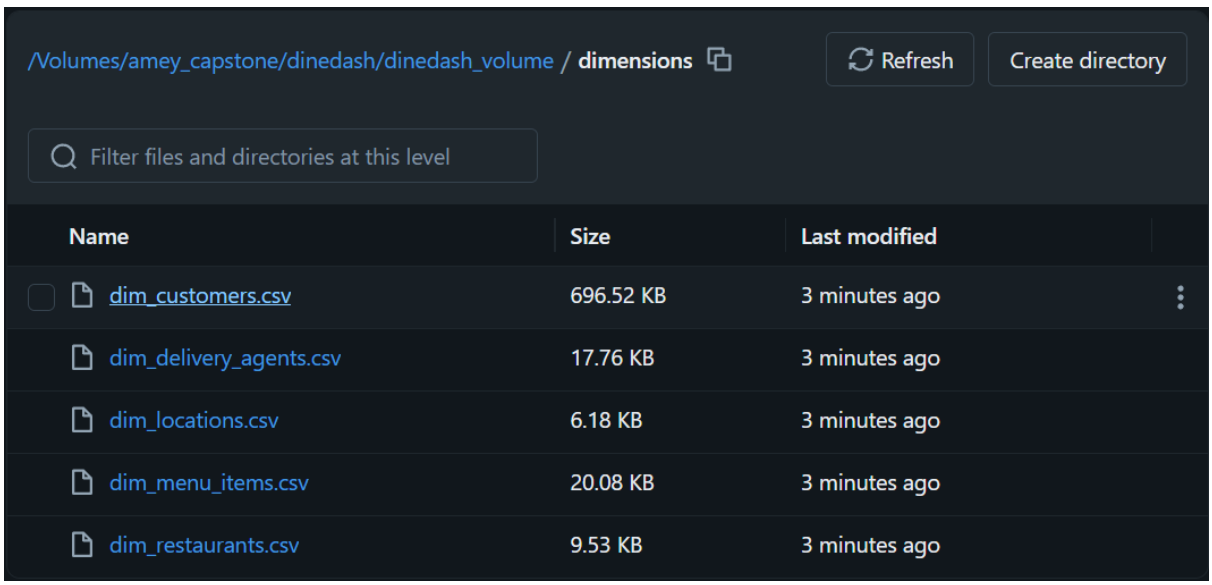
4. Menu Optimization: Identifying popular and profitable menu items helps improve menu offerings and item combinations.
5. Order Fulfillment Delays: On-time delivery can be affected by traffic, weather, and restaurant preparation delays.
6. Promotions and Discounts: Effective offers should increase order value and repeat purchases while maintaining profitability.

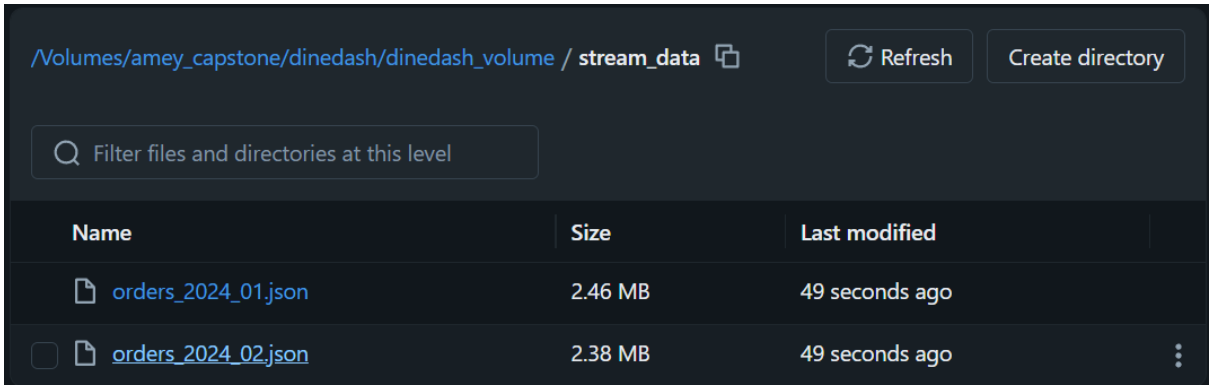
DineDash Source Data Storage Structure



A Unity Catalog Volume was created for the DineDash project . Two directories, dimensions and stream_data, were created the Volume. The dimensions directory is used to store the customer, delivery-agent, location, menu-item, and restaurant datasets, while stream_data is used to store the monthly 2024 order JSON files. This structure provides a clear separation between dimension data and order data used for incremental processing.

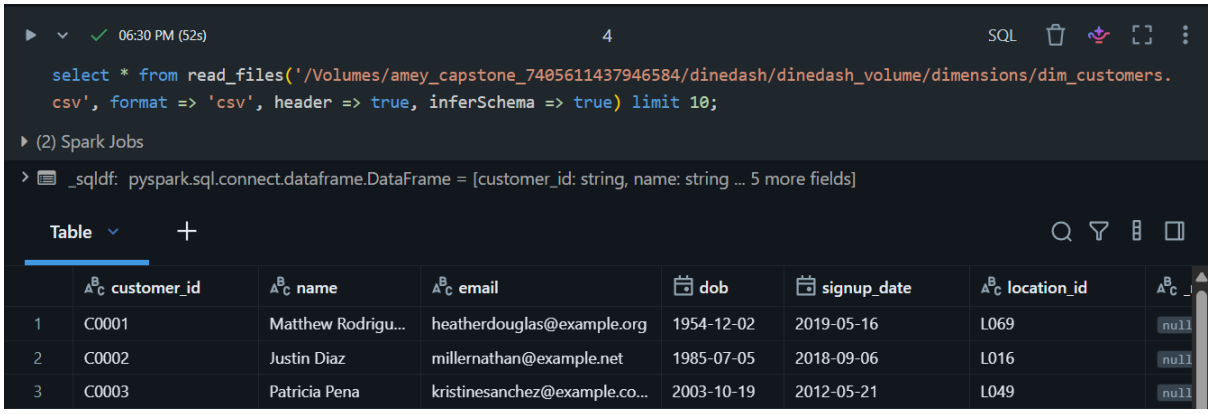
Initial DineDash Source Data Upload

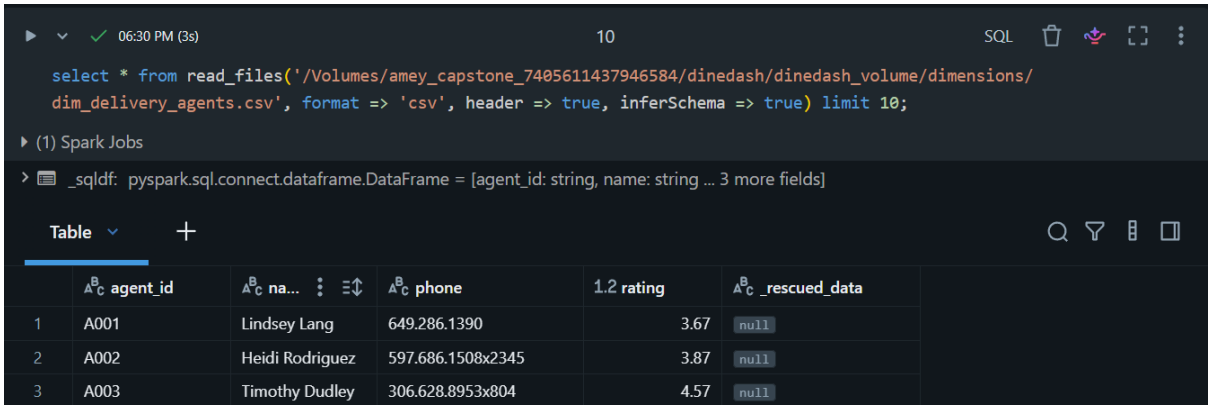




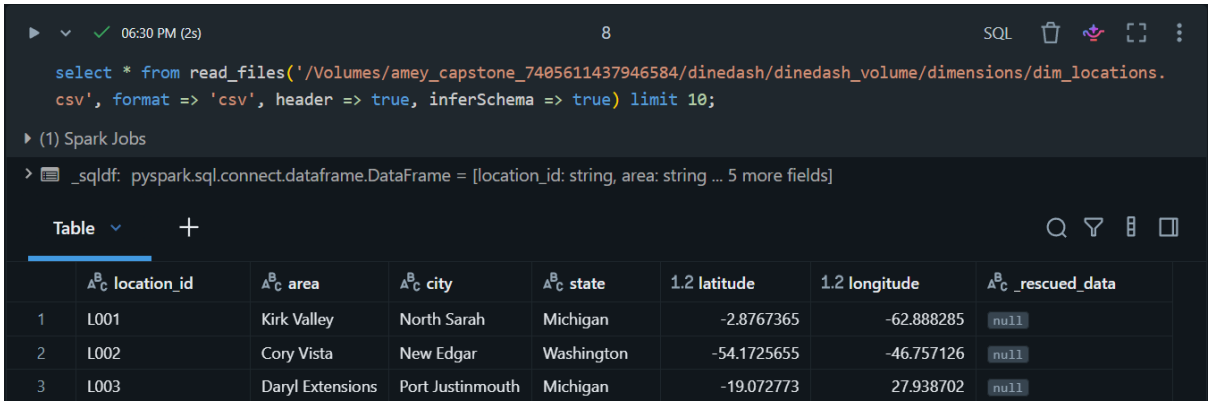
The initial source-data load contains the five-dimension datasets and the January and February 2024 order datasets. The remaining monthly order files are retained for subsequent incremental ingestion as specified in the project instructions.

Initial Data Exploration









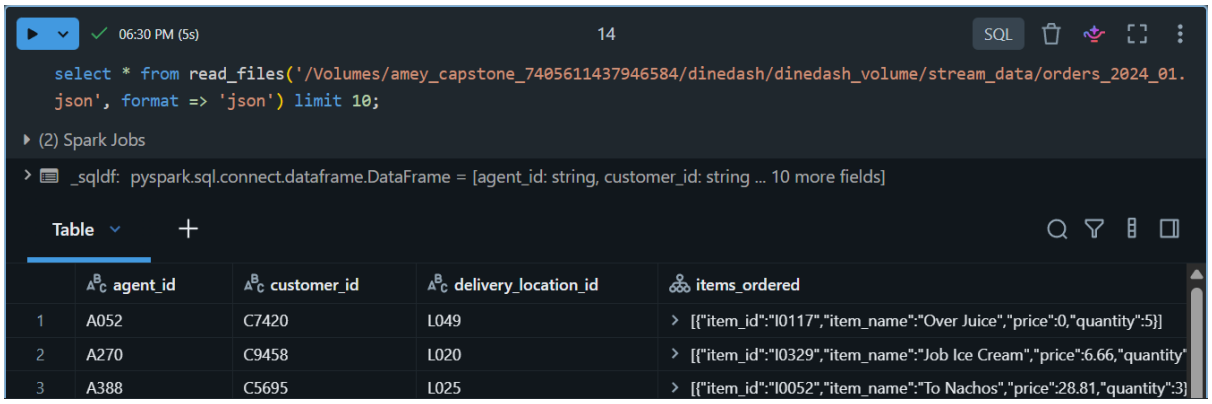
```
06:30 PM (2s)  8  SQL
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_locations.csv', format => 'csv', header => true, inferSchema => true) limit 10;
(1) Spark Jobs
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [location_id: string, area: string ... 5 more fields]
Table
```

	location_id	area	city	state	latitude	longitude	_rescued_data
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	null
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	null
3	L003	Daryl Extensions	Port Justinmouth	Michigan	-19.072773	27.938702	null



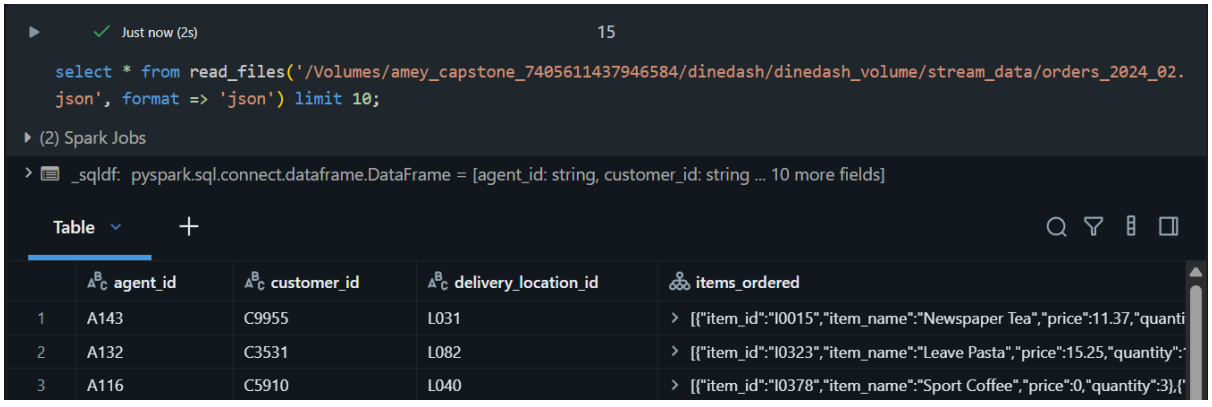
```
06:30 PM (2s)  12  SQL
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_menu_items.csv', format => 'csv', header => true, inferSchema => true) limit 10;
(1) Spark Jobs
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [item_id: string, restaurant_id: string ... 4 more fields]
Table
```

	item_id	restaurant_id	item_name	price	category	_rescued_data
1	I0001	R017	Radio Soda	17.97	Beverage	null
2	I0002	R169	Occur Pizza	27.44	Main Course	null
3	I0003	R178	Road Nachos	9.5	Appetizer	null



```
06:30 PM (5s)  14  SQL
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/stream_data/orders_2024_01.json', format => 'json') limit 10;
(2) Spark Jobs
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, customer_id: string ... 10 more fields]
Table
```

	agent_id	customer_id	delivery_location_id	items_ordered
1	A052	C7420	L049	[{"item_id":"I0117","item_name":"Over Juice","price":0,"quantity":5}]
2	A270	C9458	L020	[{"item_id":"I0329","item_name":"Job Ice Cream","price":6.66,"quantity"
3	A388	C5695	L025	[{"item_id":"I0052","item_name":"To Nachos","price":28.81,"quantity":3}



```
Just now (2s)  15
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/stream_data/orders_2024_02.json', format => 'json') limit 10;
(2) Spark Jobs
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [agent_id: string, customer_id: string ... 10 more fields]
Table
```

	agent_id	customer_id	delivery_location_id	items_ordered
1	A143	C9955	L031	[{"item_id":"I0015","item_name":"Newspaper Tea","price":11.37,"quanti
2	A132	C3531	L082	[{"item_id":"I0323","item_name":"Leave Pasta","price":15.25,"quantity":
3	A116	C5910	L040	[{"item_id":"I0378","item_name":"Sport Coffee","price":0,"quantity":3},{

The source datasets were queried directly from the DineDash Volume before table creation. Ten sample records were displayed for the dimension datasets and order data to inspect the available columns, inferred data types, and nested order-item structure. The order dataset contains an items_ordered array, which is used later for item-level transformations and analysis.

DineDash Temporary Views

```
06:30 PM (27s)                                3                                SQL
create or replace temp view customers_tmp_view as
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_customers.
csv', format => 'csv', header => true, inferSchema => true);
(1) Spark Jobs
OK
```

```
06:30 PM (12s)                                5                                SQL
create or replace temp view restaurants_tmp_view as
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_restaurants.
csv', format => 'csv', header => true, inferSchema => true);
(1) Spark Jobs
OK
```

```
06:30 PM (2s)                                 7                                SQL
create or replace temp view locations_tmp_view as
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_locations.
csv', format => 'csv', header => true, inferSchema => true);
(1) Spark Jobs
OK
```

```
06:30 PM (2s)                                 9
create or replace temp view delivery_agents_tmp_view as
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/
dim_delivery_agents.csv', format => 'csv', header => true, inferSchema => true);
(1) Spark Jobs
OK
```

```
06:30 PM (1s)                                 11                               SQL
create or replace temp view menu_items_tmp_view as
select * from read_files('/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_menu_items.
csv', format => 'csv', header => true, inferSchema => true);
(1) Spark Jobs
OK
```

```
Just now (1s)                                 12                               SQL
show views;
```

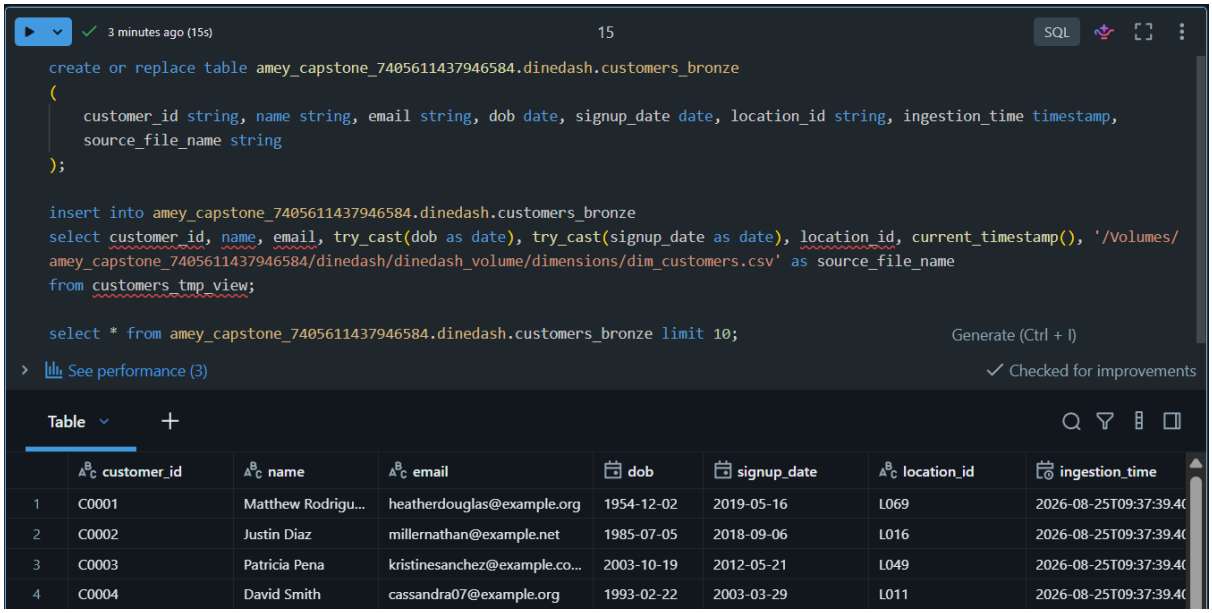
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [namespace: string, viewName: string ... 3 more fields]

Table

	namespace	viewName	isTemporary	isMaterialized	isMetric
1		_sqldf	true	false	false
2		customers_tmp_view	true	false	false
3		delivery_agents_tmp_vi...	true	false	false
4		locations_tmp_view	true	false	false
5		menu_items_tmp_view	true	false	false
6		restaurants_tmp_view	true	false	false

Temporary views were created for the five-dimension source datasets using the read_files() table-valued function with schema inference. These views provide a temporary SQL representation of the source files and are used as the starting point for creating the raw Bronze Delta tables.

Bronze tables

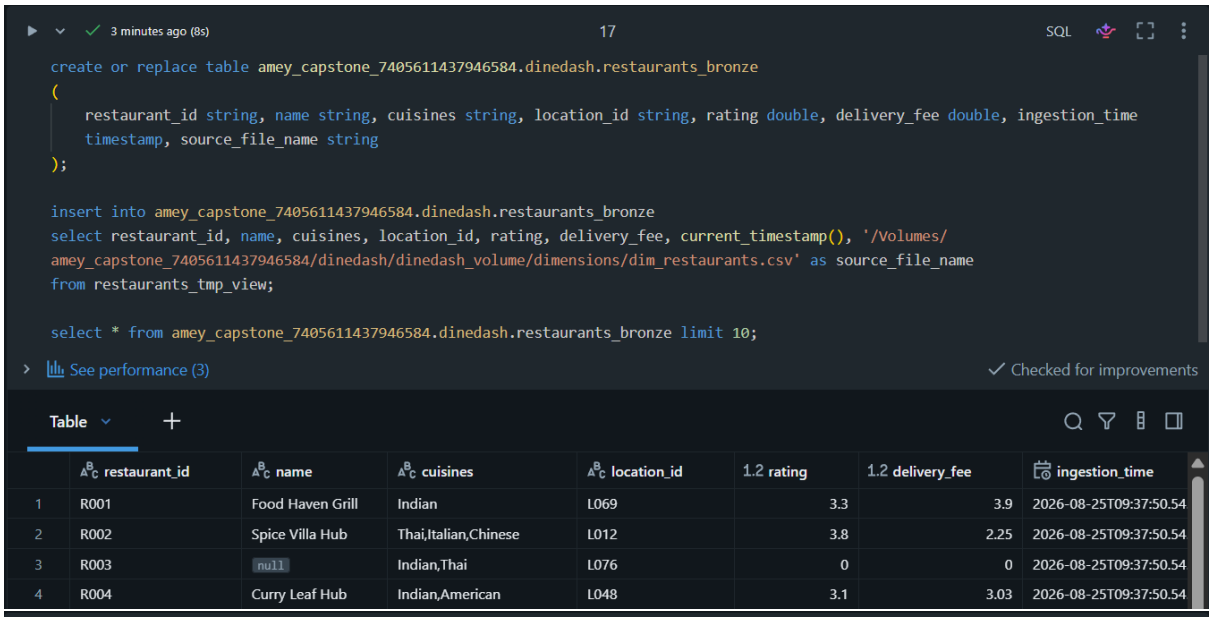


```
create or replace table amey_capstone_7405611437946584.dinedash.customers_bronze
(
    customer_id string, name string, email string, dob date, signup_date date, location_id string, ingestion_time timestamp,
    source_file_name string
);

insert into amey_capstone_7405611437946584.dinedash.customers_bronze
select customer_id, name, email, try_cast(dob as date), try_cast(signup_date as date), location_id, current_timestamp(), '/Volumes/
amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_customers.csv' as source_file_name
from customers_tmp_view;

select * from amey_capstone_7405611437946584.dinedash.customers_bronze limit 10;
```

	customer_id	name	email	dob	signup_date	location_id	ingestion_time
1	C0001	Matthew Rodrigu...	heatherdouglas@example.org	1954-12-02	2019-05-16	L069	2026-08-25T09:37:39.4(
2	C0002	Justin Diaz	millernathan@example.net	1985-07-05	2018-09-06	L016	2026-08-25T09:37:39.4(
3	C0003	Patricia Pena	kristinesanchez@example.co...	2003-10-19	2012-05-21	L049	2026-08-25T09:37:39.4(
4	C0004	David Smith	cassandra07@example.org	1993-02-22	2003-03-29	L011	2026-08-25T09:37:39.4(

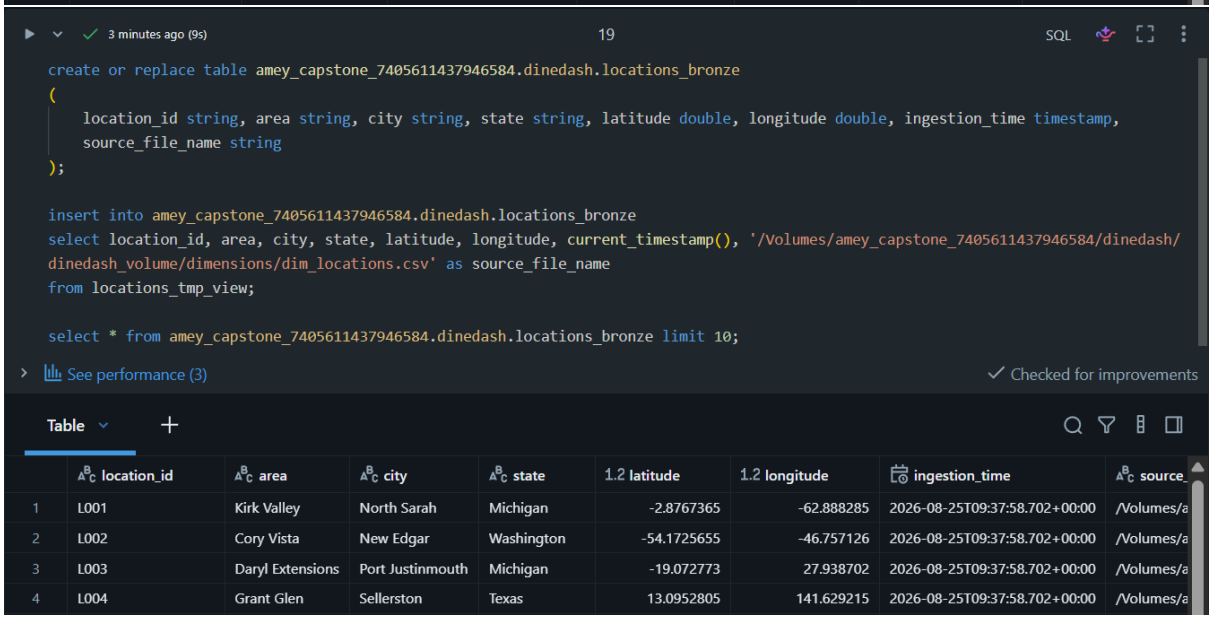


```
create or replace table amey_capstone_7405611437946584.dinedash.restaurants_bronze
(
    restaurant_id string, name string, cuisines string, location_id string, rating double, delivery_fee double, ingestion_time
    timestamp, source_file_name string
);

insert into amey_capstone_7405611437946584.dinedash.restaurants_bronze
select restaurant_id, name, cuisines, location_id, rating, delivery_fee, current_timestamp(), '/Volumes/
amey_capstone_7405611437946584/dinedash/dinedash_volume/dimensions/dim_restaurants.csv' as source_file_name
from restaurants_tmp_view;

select * from amey_capstone_7405611437946584.dinedash.restaurants_bronze limit 10;
```

	restaurant_id	name	cuisines	location_id	rating	delivery_fee	ingestion_time
1	R001	Food Haven Grill	Indian	L069	3.3	3.9	2026-08-25T09:37:50.54
2	R002	Spice Villa Hub	Thai,Italian,Chinese	L012	3.8	2.25	2026-08-25T09:37:50.54
3	R003	null	Indian,Thai	L076	0	0	2026-08-25T09:37:50.54
4	R004	Curry Leaf Hub	Indian,American	L048	3.1	3.03	2026-08-25T09:37:50.54



```
create or replace table amey_capstone_7405611437946584.dinedash.locations_bronze
(
    location_id string, area string, city string, state string, latitude double, longitude double, ingestion_time timestamp,
    source_file_name string
);

insert into amey_capstone_7405611437946584.dinedash.locations_bronze
select location_id, area, city, state, latitude, longitude, current_timestamp(), '/Volumes/amey_capstone_7405611437946584/dinedash/
dinedash_volume/dimensions/dim_locations.csv' as source_file_name
from locations_tmp_view;

select * from amey_capstone_7405611437946584.dinedash.locations_bronze limit 10;
```

	location_id	area	city	state	latitude	longitude	ingestion_time	source_
1	L001	Kirk Valley	North Sarah	Michigan	-2.8767365	-62.888285	2026-08-25T09:37:58.702+00:00	/Volumes/a
2	L002	Cory Vista	New Edgar	Washington	-54.1725655	-46.757126	2026-08-25T09:37:58.702+00:00	/Volumes/a
3	L003	Daryl Extensions	Port Justinmouth	Michigan	-19.072773	27.938702	2026-08-25T09:37:58.702+00:00	/Volumes/a
4	L004	Grant Glen	Sellerston	Texas	13.0952805	141.629215	2026-08-25T09:37:58.702+00:00	/Volumes/a

4 minutes ago (6s) 21 SQL

```
create or replace table amey_capstone_7405611437946584.dinedash.delivery_agents_bronze
(
    agent_id string, name string, phone string, rating double, ingestion_time timestamp, source_file_name string
);

insert into amey_capstone_7405611437946584.dinedash.delivery_agents_bronze
select agent_id, name, phone, rating, current_timestamp(), '/Volumes/amey_capstone_7405611437946584/dinedash/dinedash_volume/
dimensions/dim_delivery_agents.csv' as source_file_name
from delivery_agents_tmp_view;

select * from amey_capstone_7405611437946584.dinedash.delivery_agents_bronze limit 10;
```

See performance (3) Checked for improvements

Table

	agent_id	name	phone	rating	ingestion_time	source_file_name
1	A001	Lindsey Lang	649.286.1390	3.67	2026-08-25T09:38:06.519+00:00	/Volumes/amey_capstone_7405611437946584/
2	A002	Heidi Rodriguez	597.686.1508x2345	3.87	2026-08-25T09:38:06.519+00:00	/Volumes/amey_capstone_7405611437946584/
3	A003	Timothy Dudley	306.628.8953x804	4.57	2026-08-25T09:38:06.519+00:00	/Volumes/amey_capstone_7405611437946584/
4	A004	Victor Caldwell	null	0	2026-08-25T09:38:06.519+00:00	/Volumes/amey_capstone_7405611437946584/

4 minutes ago (6s) 23

```
create or replace table amey_capstone_7405611437946584.dinedash.menu_items_bronze
(
    item_id string, restaurant_id string, item_name string, price double, category string, ingestion_time timestamp,
    source_file_name string
);

insert into amey_capstone_7405611437946584.dinedash.menu_items_bronze
select item_id, restaurant_id, item_name, price, category, current_timestamp(), '/Volumes/amey_capstone_7405611437946584/dinedash/
dinedash_volume/dimensions/dim_menu_items.csv' as source_file_name
from menu_items_tmp_view;

select * from amey_capstone_7405611437946584.dinedash.menu_items_bronze limit 10;
```

See performance (3) Checked for improvements

Table

	item_id	restaurant_id	item_name	price	category	ingestion_time	source_file_name
1	I0001	R017	Radio Soda	17.97	Beverage	2026-08-25T09:38:13.053+00:00	/Volumes/amey_capstone
2	I0002	R169	Occur Pizza	27.44	Main Course	2026-08-25T09:38:13.053+00:00	/Volumes/amey_capstone
3	I0003	R178	Road Nachos	9.5	Appetizer	2026-08-25T09:38:13.053+00:00	/Volumes/amey_capstone
4	I0004	R032	Detail Juice	5.98	Beverage	2026-08-25T09:38:13.053+00:00	/Volumes/amey_capstone

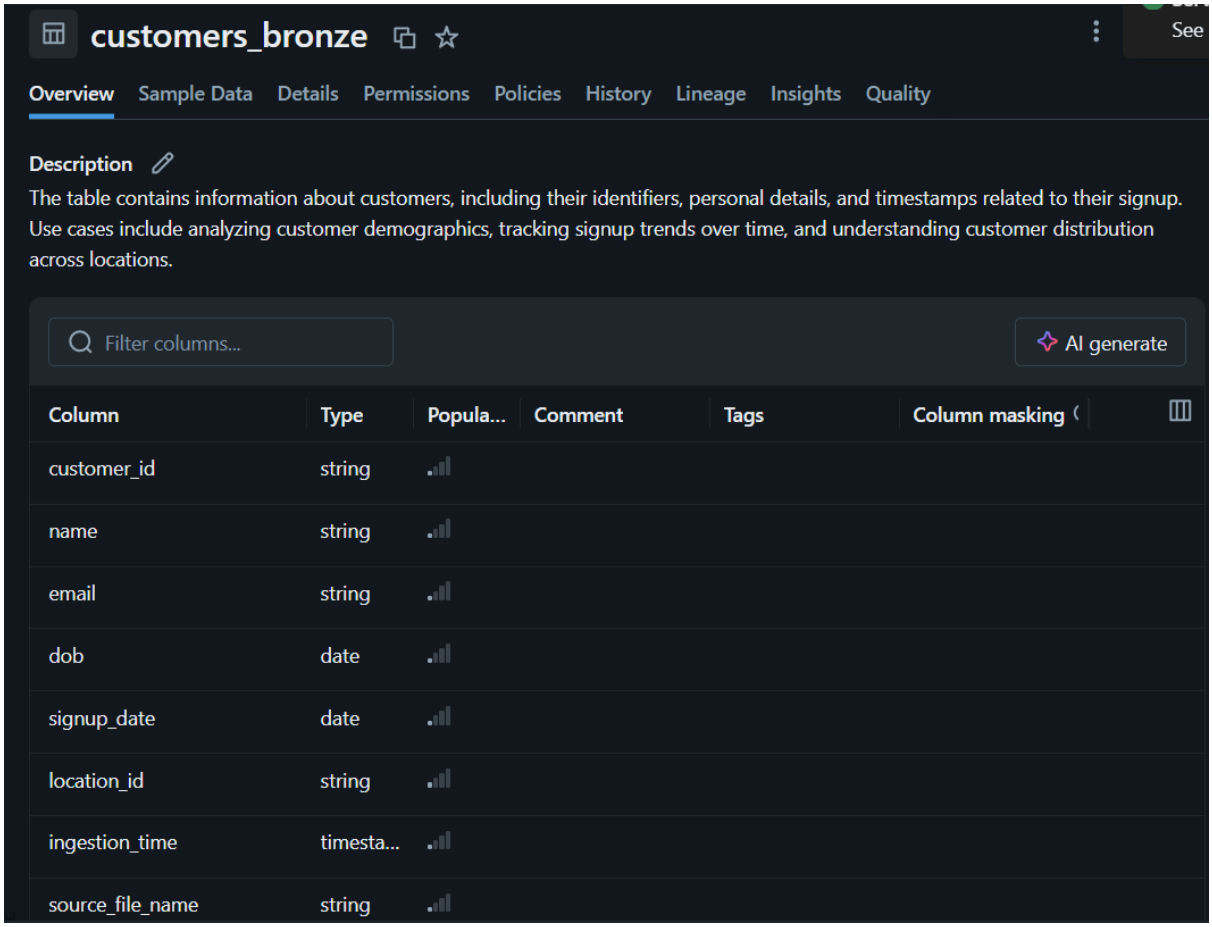
Filter tables

Tables 5 | Volumes 1 | Models 0 | Functions 0 | Secrets 0 | Services 0 | Connections 0

Sort

Name	Owner	Created at	Popularity	Comment
customers_bronze	fy26databricks5th...	Aug 21, 2026, 12:...		
delivery_agents_bro...	fy26databricks5th...	Aug 21, 2026, 12:...		
locations_bronze	fy26databricks5th...	Aug 21, 2026, 12:...		
menu_items_bronze	fy26databricks5th...	Aug 21, 2026, 12:...		
restaurants_bronze	fy26databricks5th...	Aug 21, 2026, 12:...		

Bronze table data



Raw Delta tables were created for the five-dimension datasets using manually defined schemas. The Bronze tables preserve the source attributes and additionally record the ingestion timestamp and source file name for each record.

Implementation detail: each Bronze table was first created using an explicit manual schema and then populated from its temporary view with INSERT INTO

DineDash Filtered Dimension Tables

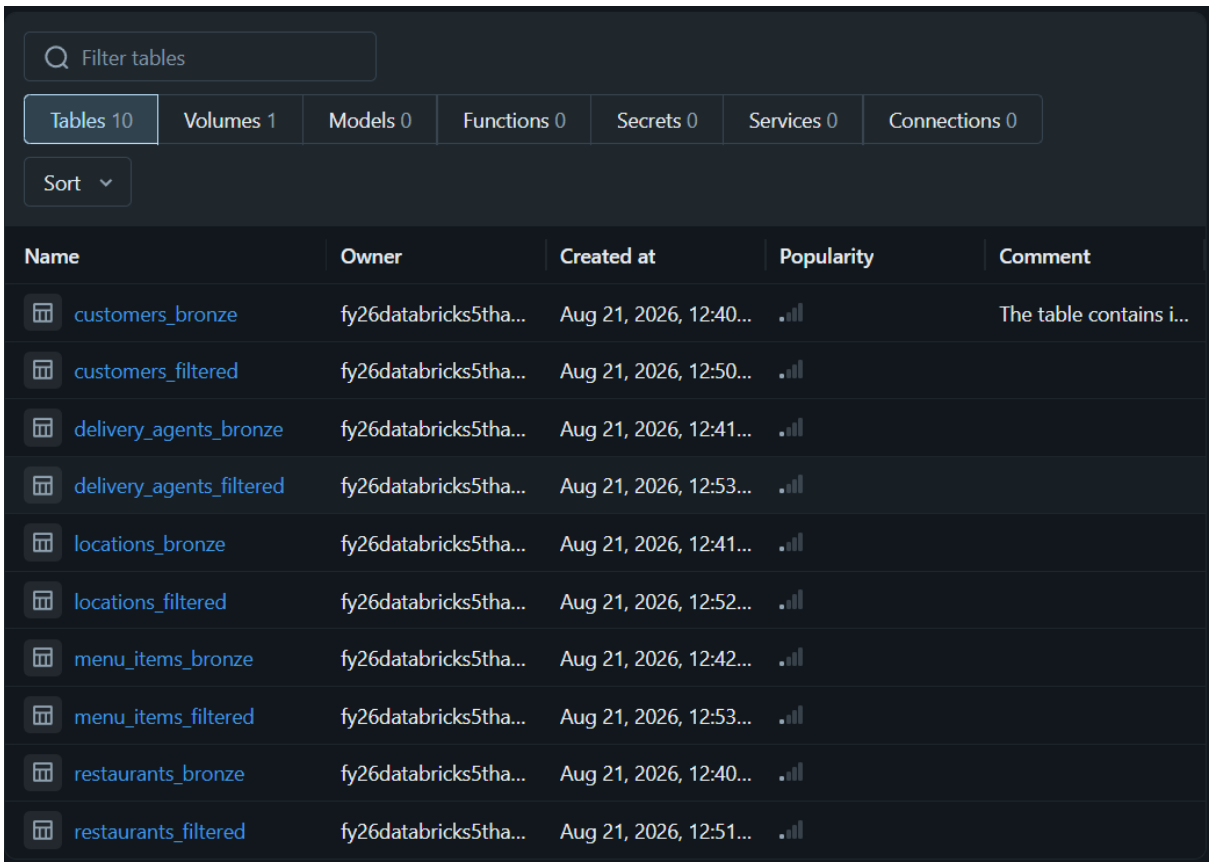
```
create or replace table amey_capstone_7405611437946584.dinedash.customers_filtered as
select *
from amey_capstone_7405611437946584.dinedash.customers_bronze
where customer_id is not null and location_id is not null and signup_date is not null and dob is not null and email
is not null and name is not null;
```

```
create or replace table amey_capstone_7405611437946584.dinedash.restaurants_filtered as
select *
from amey_capstone_7405611437946584.dinedash.restaurants_bronze
where restaurant_id is not null and name is not null and cuisines is not null and location_id is not null and rating
is not null and delivery_fee is not null;
```

```
create or replace table amey_capstone_7405611437946584.dinedash.locations_filtered as
select *
from amey_capstone_7405611437946584.dinedash.locations_bronze
where location_id is not null and area is not null and city is not null and state is not null and latitude is not
null and longitude is not null;
```

```sql
create or replace table amey_capstone_7405611437946584.dinedash.delivery_agents_filtered as
select *
from amey_capstone_7405611437946584.dinedash.delivery_agents_bronze
where agent_id is not null and name is not null and phone is not null and rating is not null;
```

```sql
create or replace table amey_capstone_7405611437946584.dinedash.menu_items_filtered as
select *
from amey_capstone_7405611437946584.dinedash.menu_items_bronze
where item_id is not null and restaurant_id is not null and item_name is not null and price is not null and category
is not null;
```



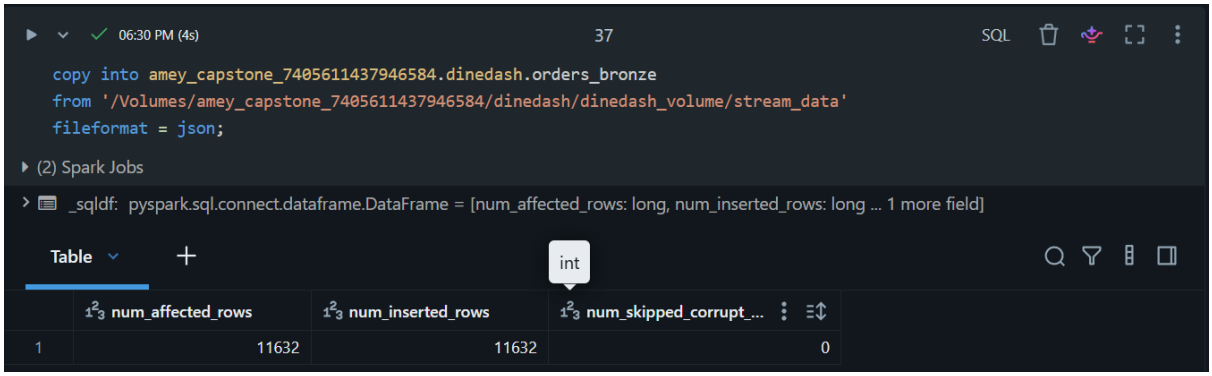
Dimension Data Quality Validation

```sql
select 'customers' as dataset,
  (select count(*) from amey_capstone_7405614862286237.dinedash.customers_bronze) as bronze_count,
  (select count(*) from amey_capstone_7405614862286237.dinedash.customers_filtered) as filtered_count
union all
select 'restaurants',
  (select count(*) from amey_capstone_7405614862286237.dinedash.restaurants_bronze),
  (select count(*) from amey_capstone_7405614862286237.dinedash.restaurants_filtered)
union all
select 'locations',
  (select count(*) from amey_capstone_7405614862286237.dinedash.locations_bronze),
  (select count(*) from amey_capstone_7405614862286237.dinedash.locations_filtered)
union all
select 'delivery_agents',
  (select count(*) from amey_capstone_7405614862286237.dinedash.delivery_agents_bronze),
  (select count(*) from amey_capstone_7405614862286237.dinedash.delivery_agents_filtered)
union all
select 'menu_items',
  (select count(*) from amey_capstone_7405614862286237.dinedash.menu_items_bronze),
  (select count(*) from amey_capstone_7405614862286237.dinedash.menu_items_filtered);
```

	dataset	bronze_count	filtered_count
1	customers	10050	9999
2	restaurants	200	150
3	locations	100	100
4	delivery_agents	450	395
5	menu_items	500	500

The Bronze dimension tables were filtered using the mandatory non-null fields specified in the project requirements. Records missing any of the required fields were excluded from the corresponding filtered tables.

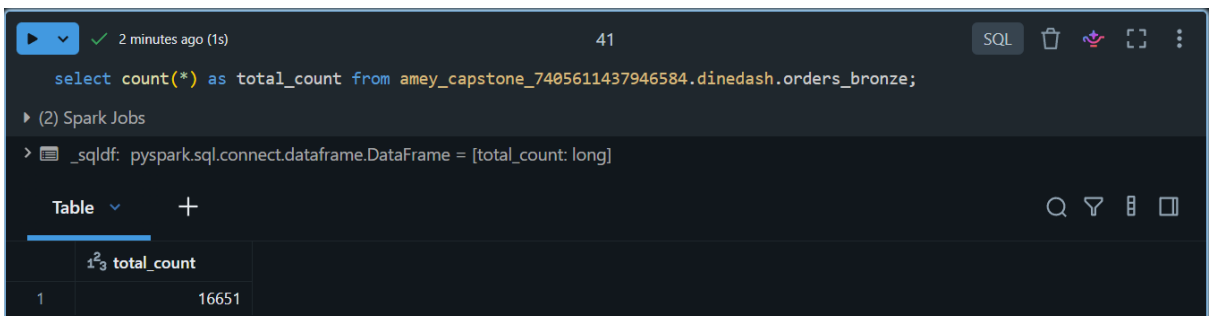
Jan+Feb Order Load Verification



March Incremental Order Load Verification



Total Count



An **orders_bronze** Delta table was created without a predefined schema. Order data was incrementally loaded from the DineDash Volume using the COPY INTO command. January 2024-February 2024 order data was loaded first and the record count was verified. March 2024 data was subsequently introduced

and loaded using the same ingestion process, with the final record count verified. This demonstrates the incremental monthly ingestion process required for the order data.

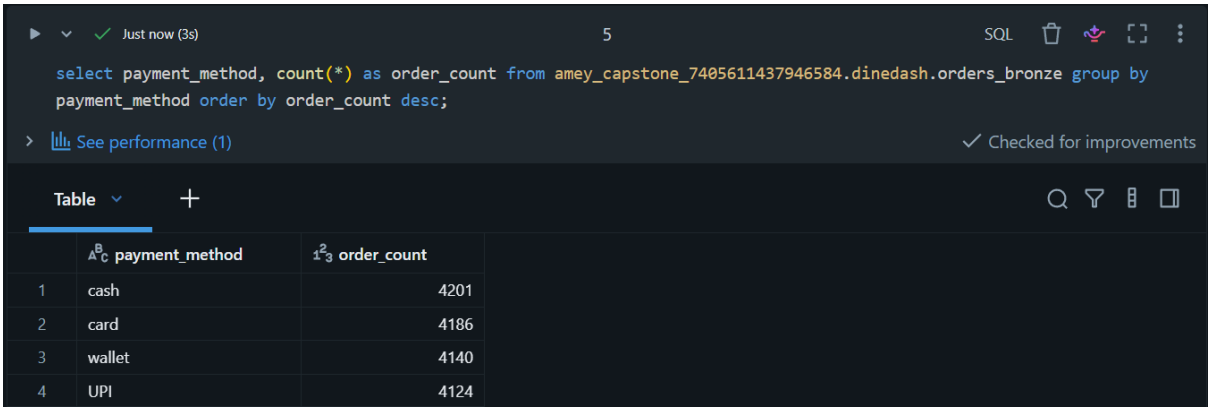
12 Queries

Q1. Total number of orders placed.



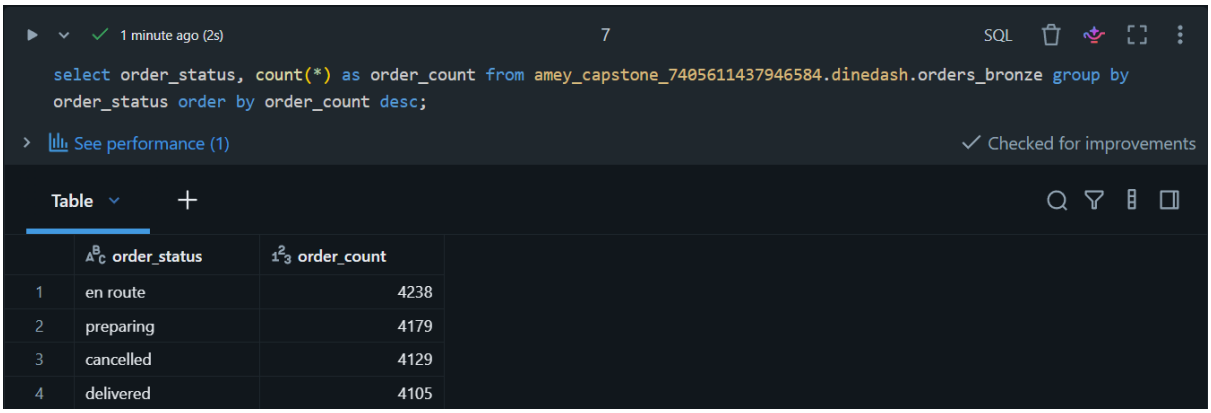
The total number of orders placed was calculated by counting all records in the **orders_bronze** table.

Q2. Distribution of orders by payment method.



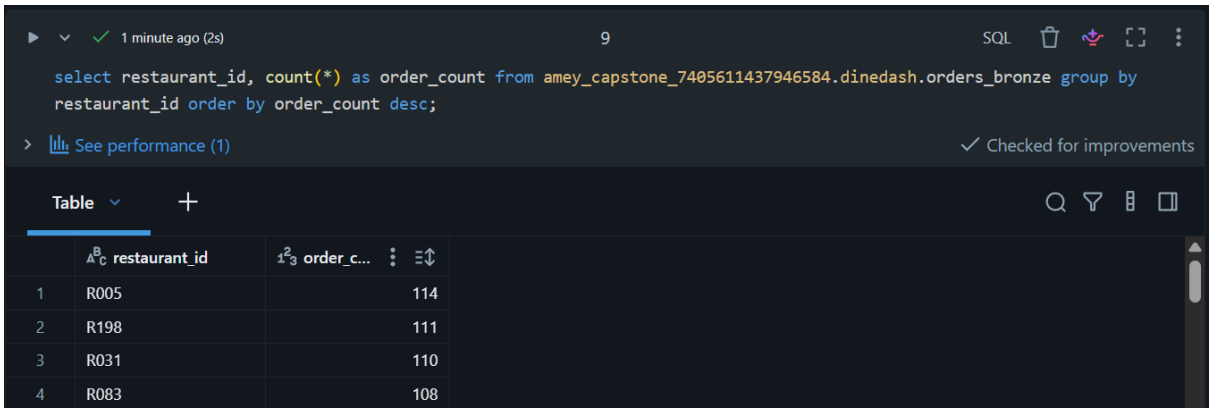
The distribution of orders by payment method was determined by grouping the orders according to payment_method and counting the number of orders in each group.

Q3. Distribution of order status (e.g., delivered, preparing, cancelled).



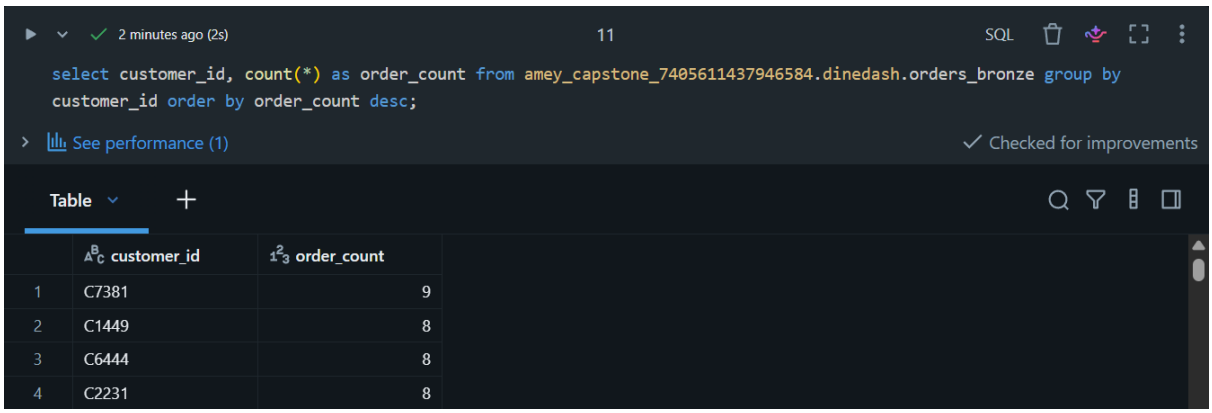
The distribution of orders by status was determined by grouping the order records according to order_status and counting the number of orders in each status.

Q4. Orders per restaurant (volume of business).



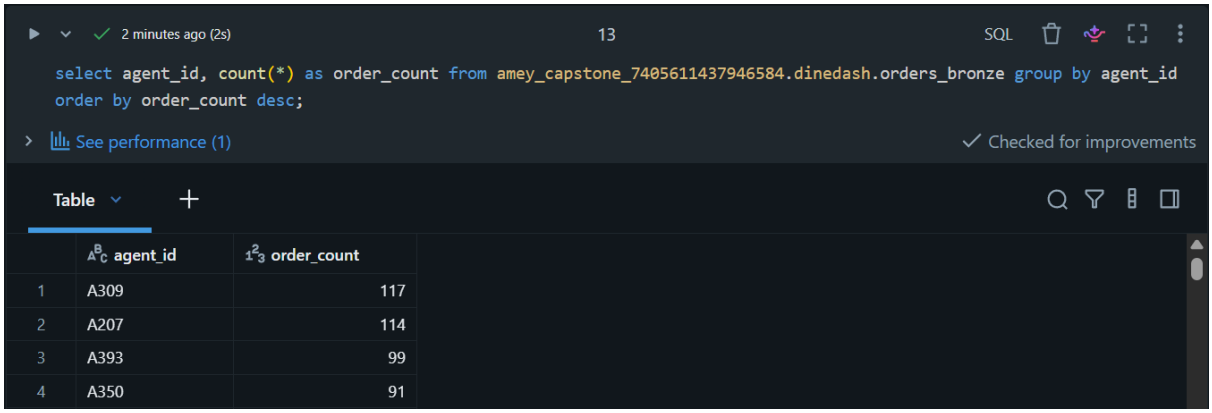
The order volume for each restaurant was calculated by grouping orders according to restaurant_id and counting the number of orders associated with each restaurant.

Q5. Orders per customer (loyalty insight).



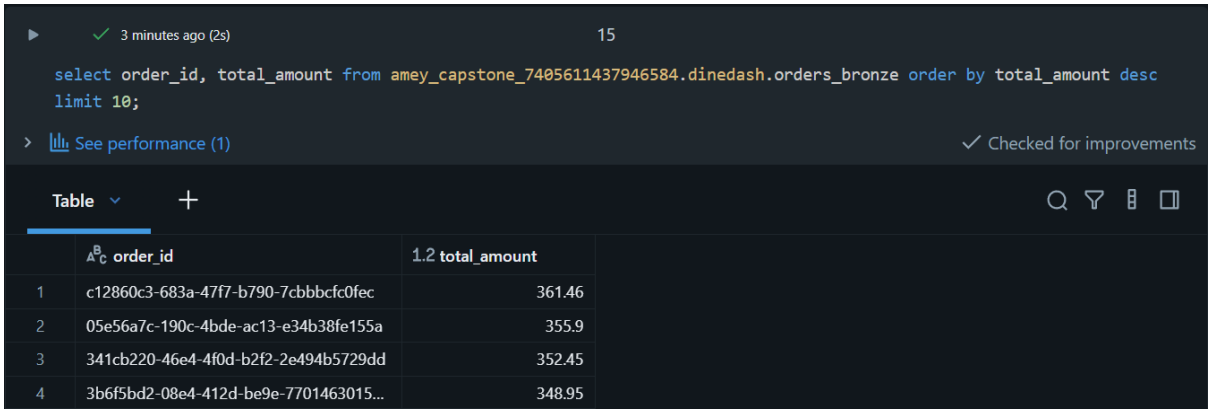
The number of orders placed by each customer was calculated by grouping the order records by customer_id and counting the associated orders. This provides an indication of customer ordering frequency.

Q6. Orders handled per delivery agent.



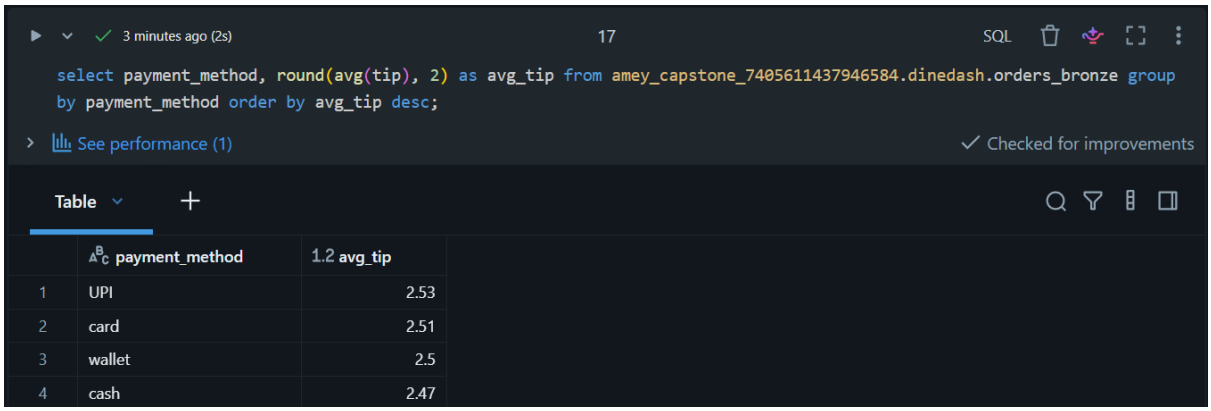
The number of orders handled by each delivery agent was calculated by grouping the orders by agent_id and counting the associated order records.

Q7. Top 10 orders by total amount.



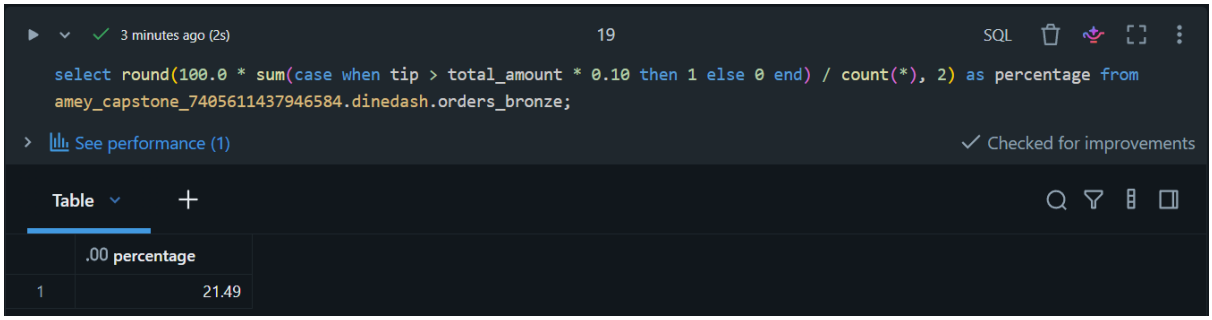
The top 10 orders by total amount were identified by sorting the orders in descending order of total_amount and selecting the first 10 records.

Q8. Average tip amount per payment method.



The average tip amount was calculated for each payment method by grouping orders according to payment_method. The resulting floating-point values were rounded up to two decimal places as specified in the project requirements.

Q9. Percentage of orders with tip > 10% of total_amount.



The percentage of orders where the tip exceeded 10% of the total order amount was calculated by comparing tip with 10% of total_amount for each order and dividing the qualifying orders by the total number of orders. The resulting percentage was rounded up to two decimal places.

Q10. Percentage of successful vs cancelled vs failed orders.

```
select order_status, round(100.0 * count(*) / sum(count(*)) over (), 2) as percentage from
amey_capstone_7405611437946584.dinedash.orders_bronze group by order_status;
```

See performance (1) — Checked for improvements

Table

	order_status	percentage
1	delivered	24.65
2	cancelled	24.80
3	preparing	25.10
4	en route	25.45

The percentage distribution of orders by status was calculated by determining the proportion of orders represented by each order_status relative to the total number of orders. The resulting percentages were rounded up to two decimal places.

Q11. Top customers (by number of orders) and top restaurants (by number of orders).

Top Customers

```
select customer_id, count(*) as order_count from amey_capstone_7405611437946584.dinedash.orders_bronze group by
customer_id order by order_count desc limit 10;
```

See performance (1) — Checked for improvements

Table

	custom...	order_count
1	C7381	9
2	C1449	8
3	C2231	8
4	C6444	8

Top Restaurants

```
select restaurant_id, count(*) as order_count from amey_capstone_7405611437946584.dinedash.orders_bronze group by
restaurant_id order by order_count desc limit 10;
```

See performance (1) — Checked for improvements

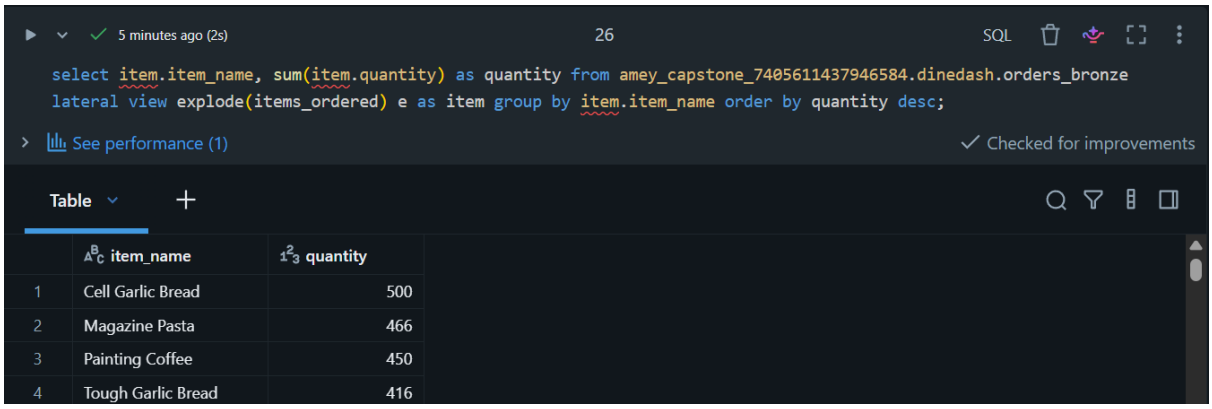
Table

	restaurant_id	order_count
1	R005	114
2	R198	111
3	R031	110
4	R131	108

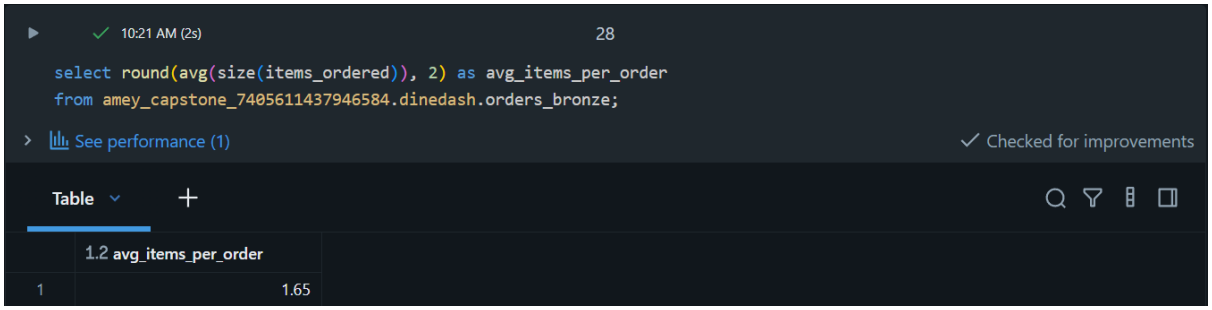
The top customers were identified by grouping orders by customer_id, counting the orders for each customer, and selecting the 10 customers with the highest order counts. Similarly, the top restaurants were identified by grouping orders by restaurant_id and selecting the 10 restaurants with the highest order counts.

Q12. Item-Level Insights (explode items_ordered):

A. Find the most frequently ordered items.



B. Find the average number of items per order.



C. Find item revenue contribution per restaurant.



The explode() function converts each item in the items_ordered array into a separate row. This allows us to calculate the most frequently ordered items, average number of items per order, and item-level revenue contribution for each restaurant.

15 Queries

Q1. Find the preferred restaurant names for each customer.

```sql
with customer_restaurant_counts as (
  select o.customer_id, r.name as restaurant_name, count(*) as order_count
  from amey_capstone_7405611437946584.dinedash.orders_bronze o
  join amey_capstone_7405611437946584.dinedash.restaurants_filtered r
    on o.restaurant_id = r.restaurant_id
  group by o.customer_id, r.name
),
ranked as (
  select *,
         row_number() over (partition by customer_id order by order_count desc) as rn
  from customer_restaurant_counts
)
select customer_id, restaurant_name, order_count
from ranked
where rn = 1
order by customer_id;
```

See performance (1) ✓ Checked for improvements

Table

	customer_id	restaurant_name	order_count
1	C0001	Spice Villa Grill	2
2	C0003	Spice Villa Grill	1
3	C0004	Tasty Bites Corner	1

Customer ordering preferences were determined by counting orders for each customer-restaurant combination and ranking the restaurants based on order frequency for each customer. The restaurant or restaurants with the highest order count were identified as the customer's preferred restaurant.

Q2. Find the top 10 areas receiving orders in each month.

▶ ✓ 10:21 AM (6s) 5

```sql
with monthly_area as (
  select date_format(to_timestamp(o.timestamp), 'yyyy-MM') as month, l.area, count(*) as order_count
  from amey_capstone_7405611437946584.dinedash.orders_bronze o
  join amey_capstone_7405611437946584.dinedash.locations_filtered l on o.delivery_location_id = l.location_id
  group by date_format(to_timestamp(o.timestamp), 'yyyy-MM'), l.area
)
select month, area, order_count
from (
  select *, row_number() over (partition by month order by order_count desc) as rn
  from monthly_area
) x
where rn <= 10
order by month, order_count desc;
```

See performance (1) ✓ Checked for improvements

Table

	month	area	order_count
1	2024-01	Rodriguez Mill	87
2	2024-01	Eric Hills	86
3	2024-01	Hill Station	80
4	2024-01	Smith Estates	79

Orders were grouped by delivery area and month using the order timestamp. The areas were ranked within each month according to order volume, and the top 10 areas for each month were selected.

Q3. Find all successfully delivered orders that were paid using a card, to analyze payment preferences and delivery success rates.



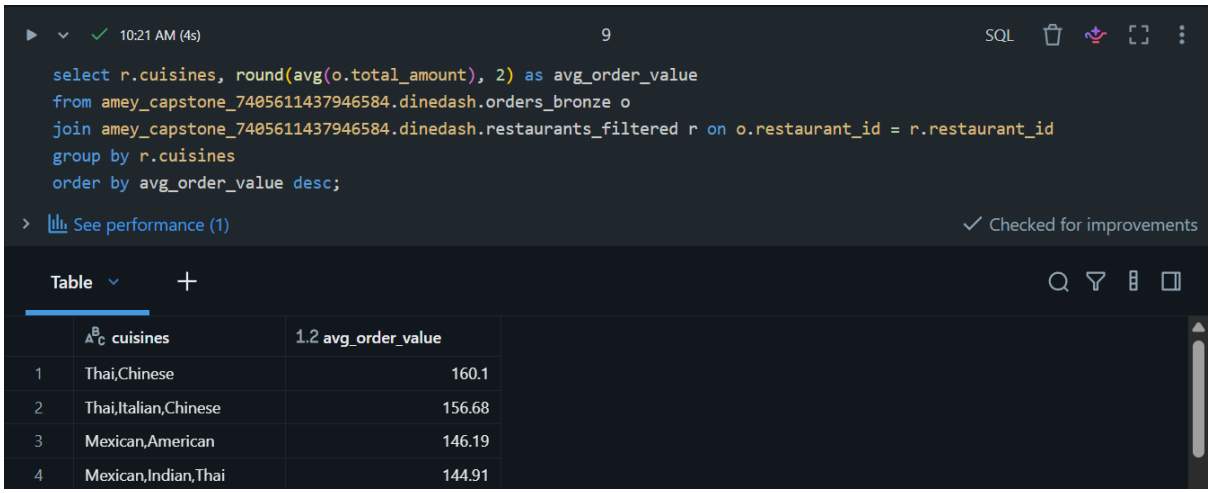
```
select order_id, customer_id, restaurant_id, total_amount
from amey_capstone_7405611437946584.dinedash.orders_bronze
where lower(order_status) = 'delivered' and lower(payment_method) = 'card';
```

10:21 AM (3s) · 7 · SQL · See performance (1) · Checked for improvements · Table

	order_id	customer_id	restaurant_id	total_amount
1	e293e41d-e335-4d6b-8504-34f96140e660	C5605	R006	21.2
2	c8bd7ee7-a451-447a-811b-fd69aff36062	C7525	R053	75.97
3	e71e1b46-c026-4913-9b58-870fde58e72b	C1909	R047	113.32
4	ede3312f-d227-42a5-a1d0-d191999abbd1	C3246	R136	112.3

Orders were filtered to identify successfully delivered orders where the payment method was card. The result provides the order, customer, restaurant, and amount details for this combination of delivery status and payment preference.

Q4. Which cuisines generate the highest average order value?
Hint: Join orders, restaurants, group by cuisine, and calculate average total_amount.



```
select r.cuisines, round(avg(o.total_amount), 2) as avg_order_value
from amey_capstone_7405611437946584.dinedash.orders_bronze o
join amey_capstone_7405611437946584.dinedash.restaurants_filtered r on o.restaurant_id = r.restaurant_id
group by r.cuisines
order by avg_order_value desc;
```

10:21 AM (4s) · 9 · SQL · See performance (1) · Checked for improvements · Table

	cuisines	avg_order_value
1	Thai,Chinese	160.1
2	Thai,Italian,Chinese	156.68
3	Mexican,American	146.19
4	Mexican,Indian,Thai	144.91

Orders were joined with restaurant information and grouped by cuisine. The average total_amount was calculated for each cuisine to identify cuisines generating the highest average order value.

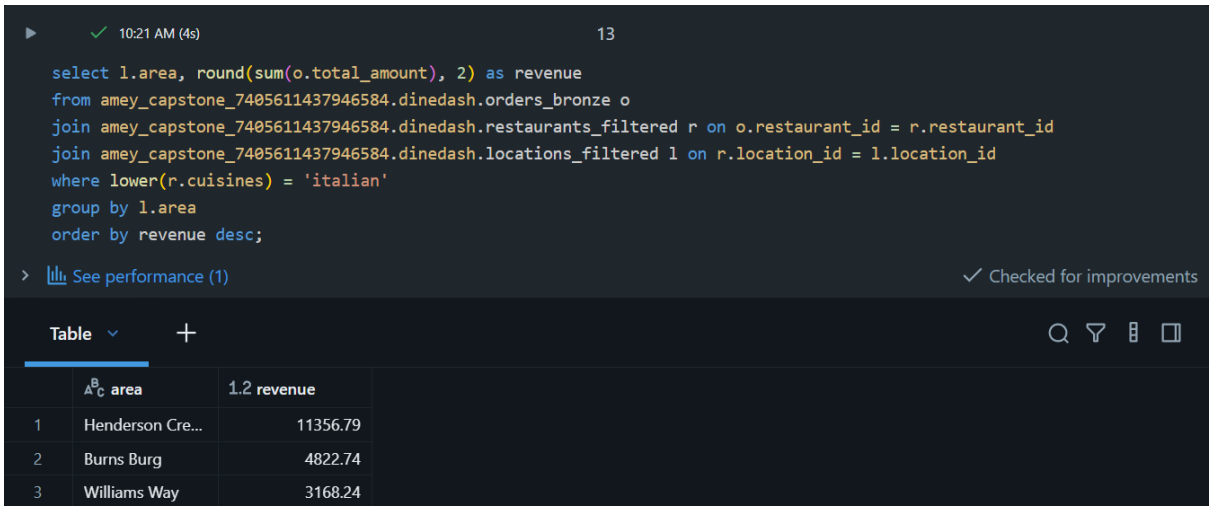
Q5. Which delivery agents have delivered the most high-value orders (above $50)?
Hint: Filter orders for total_amount > 50, join with agent, group by agent_id, and count orders.



	agent_id	name	high_value_orders
1	A309	Christopher Chase	213
2	A207	Christopher Williams	171
3	A393	Justin Lam	118
4	A350	Charlene Morgan	112

Orders with a total_amount greater than $50 were filtered and grouped by delivery agent. The number of high-value orders handled by each agent was then counted and sorted in descending order.

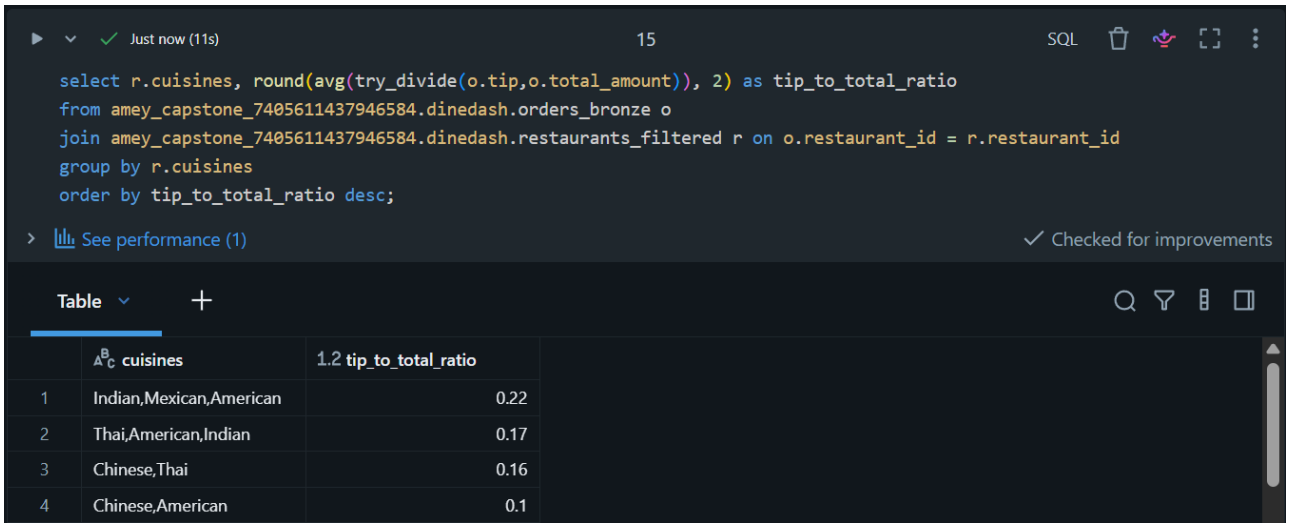
Q6. Which cities (area_name) are generating the most revenue for Italian cuisine?
Hint: Join orders, restaurants, locations, filter cuisine = 'Italian', group by area_name, and sum total_amount.



	area	revenue
1	Henderson Cre...	11356.79
2	Burns Burg	4822.74
3	Williams Way	3168.24

Orders were joined with restaurant and location information. Orders from Italian restaurants were filtered and grouped by area, and the total revenue was calculated by summing total_amount for each area.

Q7. What is the tip-to-total ratio by cuisine type?
Hint: Join orders, restaurants, group by cuisine, and calculate average tip / total_amount.



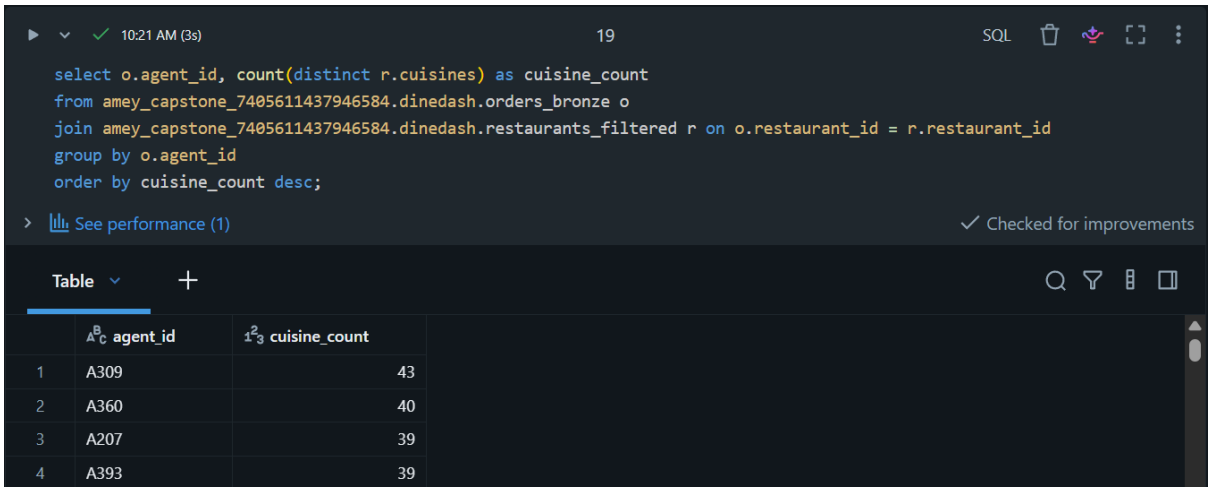
Orders were joined with restaurant information and grouped by cuisine. The average ratio of tip to total order amount was calculated for each cuisine type.

Q8. Which menu items are most frequently ordered, and from which restaurants?
Hint: Explode items_ordered in orders, join with menu_items, group by item_name, restaurant_id, and count.



The nested items_ordered array was exploded into individual menu-item records. The items were then grouped by item name and restaurant, and their order frequency was calculated.

Q9. Which agents have delivered the most diverse cuisines?
Hint: Join orders, restaurants, group by agent_id, and count distinct cuisine.



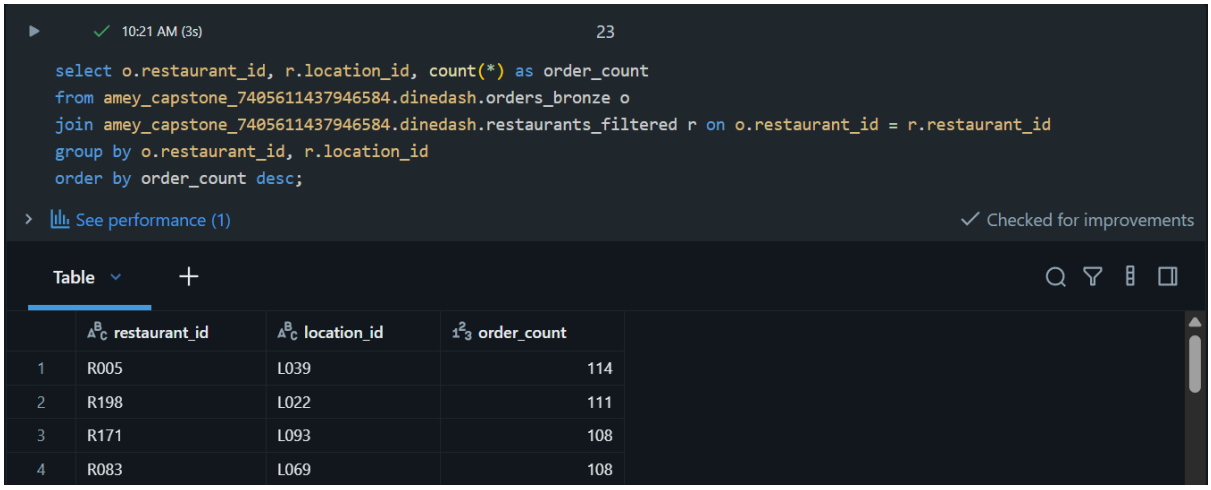
Delivery orders were joined with restaurant information and grouped by delivery agent. The number of distinct cuisines associated with each agent was calculated to identify agents handling the widest variety of cuisines.

Q10. What is the average customer lifetime value by signup year?
Hint: Join orders, customers, extract year from signup_date, group by year, sum total_amount per customer, then average total_amount per customer.



Customer lifetime value was calculated by summing the total order amount for each customer. Customers were grouped according to their signup year, and the average lifetime value was then calculated for each signup year.

Q11. Which restaurants have the highest order frequency per location?
Hint: Join orders, restaurants, locations, group by restaurant_id, location_id, and count orders.



```
select o.restaurant_id, r.location_id, count(*) as order_count
from amey_capstone_7405611437946584.dinedash.orders_bronze o
join amey_capstone_7405611437946584.dinedash.restaurants_filtered r on o.restaurant_id = r.restaurant_id
group by o.restaurant_id, r.location_id
order by order_count desc;
```

10:21 AM (3s) 23

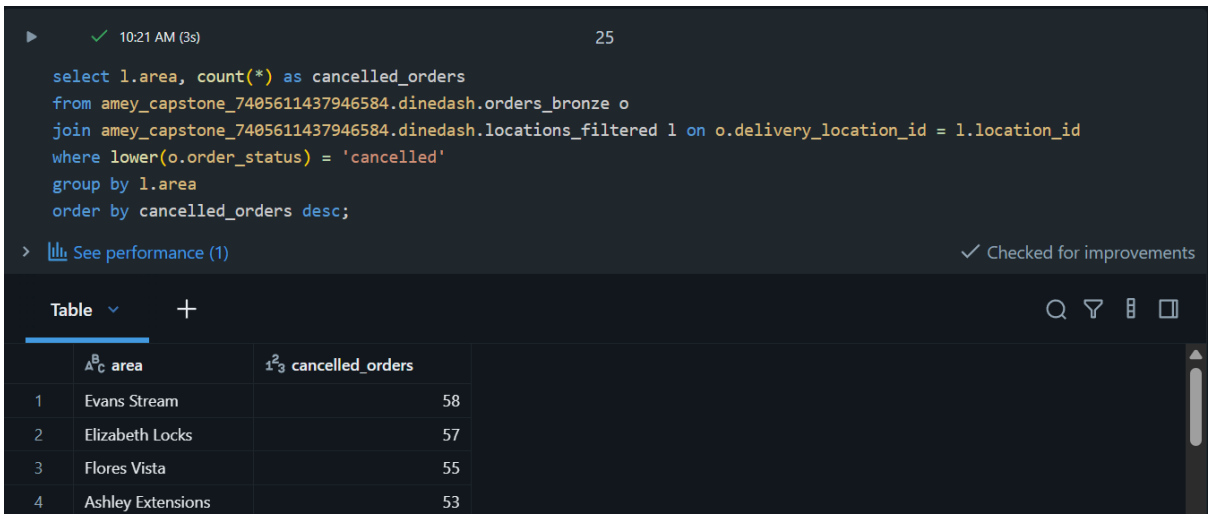
See performance (1) Checked for improvements

Table

	restaurant_id	location_id	order_count
1	R005	L039	114
2	R198	L022	111
3	R171	L093	108
4	R083	L069	108

Orders were joined with restaurant information and grouped by restaurant_id and location_id. The number of orders for each restaurant-location combination was counted and sorted in descending order to identify locations with the highest restaurant order frequency.

Q12. Which locations are prone to order cancellations?
Hint: Filter orders with order_status = 'cancelled', join locations using delivery_location_id, group by area_name, and count.



```
select l.area, count(*) as cancelled_orders
from amey_capstone_7405611437946584.dinedash.orders_bronze o
join amey_capstone_7405611437946584.dinedash.locations_filtered l on o.delivery_location_id = l.location_id
where lower(o.order_status) = 'cancelled'
group by l.area
order by cancelled_orders desc;
```

10:21 AM (3s) 25

See performance (1) Checked for improvements

Table

	area	cancelled_orders
1	Evans Stream	58
2	Elizabeth Locks	57
3	Flores Vista	55
4	Ashley Extensions	53

Cancelled orders were filtered and joined with location information using delivery_location_id. The orders were grouped by the actual location area field and counted to identify areas with the highest number of cancellations.

Q13. Find the top 10 customers who have ordered from the widest variety of restaurants.
Hint: Join orders, group by customer_id, count distinct restaurant_id, sort, and pick top 10.

```
10:21 AM (3s)                                                    27                    SQL
select customer_id, count(distinct restaurant_id) as restaurant_count
from amey_capstone_7405611437946584.dinedash.orders_bronze
group by customer_id
order by restaurant_count desc
limit 10;
```

See performance (1) ✓ Checked for improvements

Table +

	customer_id	restaurant_count
1	C7381	9
2	C1449	8
3	C2231	8
4	C6444	8

The top 10 customers with the widest variety of restaurant choices were identified by counting the distinct restaurants from which each customer had placed orders and sorting the results in descending order.

Q14. Identify restaurants with frequent repeat customers.
Hint: Join orders, group by restaurant_id, customer_id, count, and filter where count > 1.

```
10:21 AM (3s)                                                    29
select restaurant_id, customer_id, count(*) as order_count
from amey_capstone_7405611437946584.dinedash.orders_bronze
group by restaurant_id, customer_id
having count(*) > 1
order by order_count desc;
```

See performance (1) ✓ Checked for improvements

Table +

	restaurant_id	customer_id	order_count
1	R006	C5296	3
2	R017	C8372	2
3	R161	C5231	2
4	R136	C9836	2

Customer orders were grouped by restaurant_id and customer_id. Combinations with more than one order were identified as repeat-customer relationships, allowing restaurants with frequent repeat customers to be identified.

Q15. Which payment methods are most preferred for high-value orders across cuisines?
Hint: Filter orders with total_amount > 40, join restaurants, group by cuisine, payment_method, and count.

```
10:21 AM (3s)                                                    31                    SQL
select r.cuisines, o.payment_method, count(*) as order_count
from amey_capstone_7405611437946584.dinedash.orders_bronze o
join amey_capstone_7405611437946584.dinedash.restaurants_filtered r on o.restaurant_id = r.restaurant_id
where o.total_amount > 40
group by r.cuisines, o.payment_method
order by r.cuisines, order_count desc;
```

See performance (1) ✓ Checked for improvements

Table

	cuisines	payment_method	order_count
1	American	card	125
2	American	UPI	122
3	American	cash	118
4	American	wallet	117
5	American,Chinese	card	46
6	American,Chinese	UPI	46
7	American,Chinese	wallet	34

Orders with a total amount greater than $40 were filtered and joined with restaurant information. The results were grouped by cuisine and payment method, and the number of high-value orders was counted to identify payment preferences across cuisines.

Preparation of Initial DLT Input Data

The initial DLT input range was prepared by staging January–June 2024 order files in the active stream_data directory.

/Volumes/amey_capstone_7405614862286237/dinedash/dinedash_volume / **stream_data** Refresh Create directory

Filter files and directories at this level

Name	Size	Last modified
orders_2024_01.json	2.46 MB	22 hours ago
orders_2024_02.json	2.38 MB	22 hours ago
orders_2024_03.json	2.09 MB	47 minutes ago
orders_2024_04.json	3.34 MB	just now
orders_2024_05.json	2.31 MB	just now
orders_2024_06.json	3.35 MB	just now

Figure 11.1 — DLT Initial Order Data Through June 2024

DLT Bronze Layer — Raw Data Ingestion

Ingest the DineDash source datasets into the pipeline's Bronze layer. Used read_files() for the Volume-based CSV/JSON sources and a streaming Bronze table for incoming orders. Six Bronze pipeline datasets were defined.

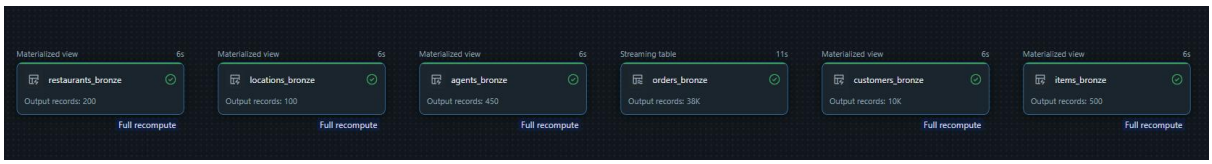
Bronze

```
2
CREATE OR REFRESH STREAMING TABLE orders_bronze AS
SELECT *
FROM STREAM(
  read_files(
    '/Volumes/amey_capstone_7405614862286237/dinedash/dinedash_volume/stream_data',
    format => 'json'
  )
);
```

```
3
CREATE LIVE TABLE restaurants_bronze AS
SELECT *
FROM read_files(
  '/Volumes/amey_capstone_7405614862286237/dinedash/dinedash_volume/dimensions/dim_restaurants.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);
```

DLT Bronze Pipeline Execution

Validate the initial Bronze ingestion pipeline. Executed the configured DLT/Lakeflow pipeline containing the Bronze ingestion definitions. Bronze datasets were processed from the Volume source files.



Silver Layer — Order Cleaning

Clean and validate incoming orders. Applied the three required expectations and exploded items_ordered. **cleaned_orders** defined as the first Silver-stage dataset.

```
9
CREATE OR REFRESH STREAMING TABLE cleaned_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_items EXPECT (size(items_ordered) > 0) ON VIOLATION DROP ROW
)
AS
SELECT
  order_id,
  timestamp AS order_ts,
  customer_id,
  restaurant_id,
  agent_id,
  delivery_location_id,
  items_ordered,
  explode(items_ordered) AS item,
  total_amount,
  tip,
  payment_method,
  order_status
FROM orders_bronze;
```

Silver Layer — **orders_silver**

Enrich cleaned orders with customer, restaurant, agent, item and location information. Joined **cleaned_orders** with the five Bronze datasets and applied the three required expectations. **orders_silver** defined as the enriched Silver dataset.

```
CREATE OR REFRESH STREAMING TABLE orders_silver (
  CONSTRAINT valid_restaurant_name EXPECT (restaurant_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_agent EXPECT (agent_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_item EXPECT (item_id IS NOT NULL) ON VIOLATION DROP ROW
)
AS
SELECT
  o.order_id,  o.order_ts,  o.customer_id,  c.name AS customer_name,  c.email, o.restaurant_id,  r.name AS restaurant_name,  r.
  cuisines,  r.rating AS restaurant_rating,  o.agent_id,  a.name AS agent_name,  a.rating AS agent_rating,  o.
  delivery_location_id,  l.city,  o.item.item_id AS item_id,  i.item_name,  i.category,  o.item.price AS item_price,  o.item.
  quantity AS item_quantity,  o.total_amount,  o.tip,  o.payment_method,  o.order_status FROM STREAM(cleaned_orders) o
LEFT JOIN customers_bronze c
  ON o.customer_id = c.customer_id
LEFT JOIN restaurants_bronze r
  ON o.restaurant_id = r.restaurant_id
LEFT JOIN agents_bronze a
  ON o.agent_id = a.agent_id
LEFT JOIN items_bronze i
  ON o.item.item_id = i.item_id
LEFT JOIN locations_bronze l
  ON o.delivery_location_id = l.location_id;
```

orders_with_agents

Enrich cleaned orders with delivery-agent information. Left joined **cleaned_orders** with agents_bronze. orders_with_agents created for agent-level analysis.

```
CREATE OR REFRESH STREAMING TABLE orders_with_agents AS
SELECT
  o.order_id,o.order_ts,o.agent_id,
  a.name AS agent_name,
  a.rating AS agent_rating,
  o.tip
FROM STREAM(cleaned_orders) o
LEFT JOIN agents_bronze a
  ON o.agent_id=a.agent_id;
```

orders_with_items

Create item-level order records. Exploded items_ordered and joined each item with items_bronze. orders_with_items created for item-level analysis.

```
CREATE OR REFRESH STREAMING TABLE orders_with_items AS
SELECT
  o.order_id,o.order_ts,o.restaurant_id,
  exploded.item_id,
  i.item_name,i.category,
  exploded.price,exploded.quantity
FROM (
  SELECT order_id,timestamp AS order_ts,restaurant_id,
         explode(items_ordered) AS exploded
  FROM STREAM(orders_bronze)
) o
LEFT JOIN items_bronze i
  ON o.exploded.item_id=i.item_id;
```

orders_with_locations

Enrich orders with delivery-location information. Joined **cleaned_orders** with **locations_bronze**. orders_with_locations created with city, state, and area.

```sql
CREATE OR REFRESH STREAMING TABLE orders_with_locations AS
SELECT
  o.order_id,o.order_ts,o.delivery_location_id,
  l.city,l.state,l.area
FROM STREAM(cleaned_orders) o
LEFT JOIN locations_bronze l
  ON o.delivery_location_id=l.location_id;
```

Gold Layer — **total_orders_per_restaurant**

Calculate total orders for each restaurant. Grouped **orders_silver** by restaurant and counted distinct orders. **total_orders_per_restaurant** defined.

```sql
CREATE LIVE TABLE total_orders_per_restaurant AS
SELECT restaurant_name,COUNT(DISTINCT order_id) AS total_orders
FROM orders_silver
GROUP BY restaurant_name;
```

Gold Layer — **top_selling_items**

Identify the top-selling menu items. Grouped **orders_silver** by item_name and summed item_quantity. Top 10 items defined in **top_selling_items**.

```sql
CREATE LIVE TABLE top_selling_items AS
SELECT item_name,SUM(item_quantity) AS total_qty_sold
FROM orders_silver
GROUP BY item_name
ORDER BY total_qty_sold DESC
LIMIT 10;
```

Gold Layer — avg_fees_by_city

Calculate city-level average restaurant rating and tip. Grouped **orders_silver** by city and calculated the two averages. avg_fees_by_city defined.

```sql
CREATE LIVE TABLE avg_fees_by_city AS
SELECT city,
       AVG(restaurant_rating) AS avg_restaurant_rating,
       AVG(tip) AS avg_tip
FROM orders_silver
GROUP BY city;
```

Gold Layer — agent_order_counts

Measure delivery-agent order volume and average rating. Grouped **orders_silver** by agent_name and calculated distinct orders and average rating. agent_order_counts defined.

```sql
CREATE LIVE TABLE agent_order_counts AS
SELECT agent_name,
       COUNT(DISTINCT order_id) AS orders_handled,
       AVG(agent_rating) AS avg_rating
FROM orders_silver
GROUP BY agent_name
ORDER BY orders_handled DESC;
```

Gold Layer — **daily_revenue_trend**

Calculate daily order revenue. Grouped **orders_silver** by order date and summed total_amount. **daily_revenue_trend** defined.

```
CREATE LIVE TABLE daily_revenue_trend AS
SELECT DATE(order_ts) AS order_date,
       SUM(total_amount) AS total_revenue
FROM orders_silver
GROUP BY DATE(order_ts)
ORDER BY order_date;
```

Gold Layer — **customer_ltv**

Calculate customer lifetime spend. Grouped **orders_silver** by customer and summed total_amount. **customer_ltv** defined.

```
CREATE LIVE TABLE customer_ltv AS
SELECT customer_id,customer_name,
       SUM(total_amount) AS lifetime_spend
FROM orders_silver
GROUP BY customer_id,customer_name
ORDER BY lifetime_spend DESC;
```

orders_with_customers

Enrich orders with customer information. Joined **cleaned_orders** with **customers_bronze**. orders_with_customers defined.

```
CREATE OR REFRESH STREAMING TABLE orders_with_customers AS
SELECT
  o.order_id,o.customer_id,
  c.name AS customer_name,
  c.location_id AS customer_location_id,
  o.total_amount,o.tip,o.payment_method,o.order_status
FROM STREAM(cleaned_orders) o
LEFT JOIN customers_bronze c
  ON o.customer_id=c.customer_id;
```

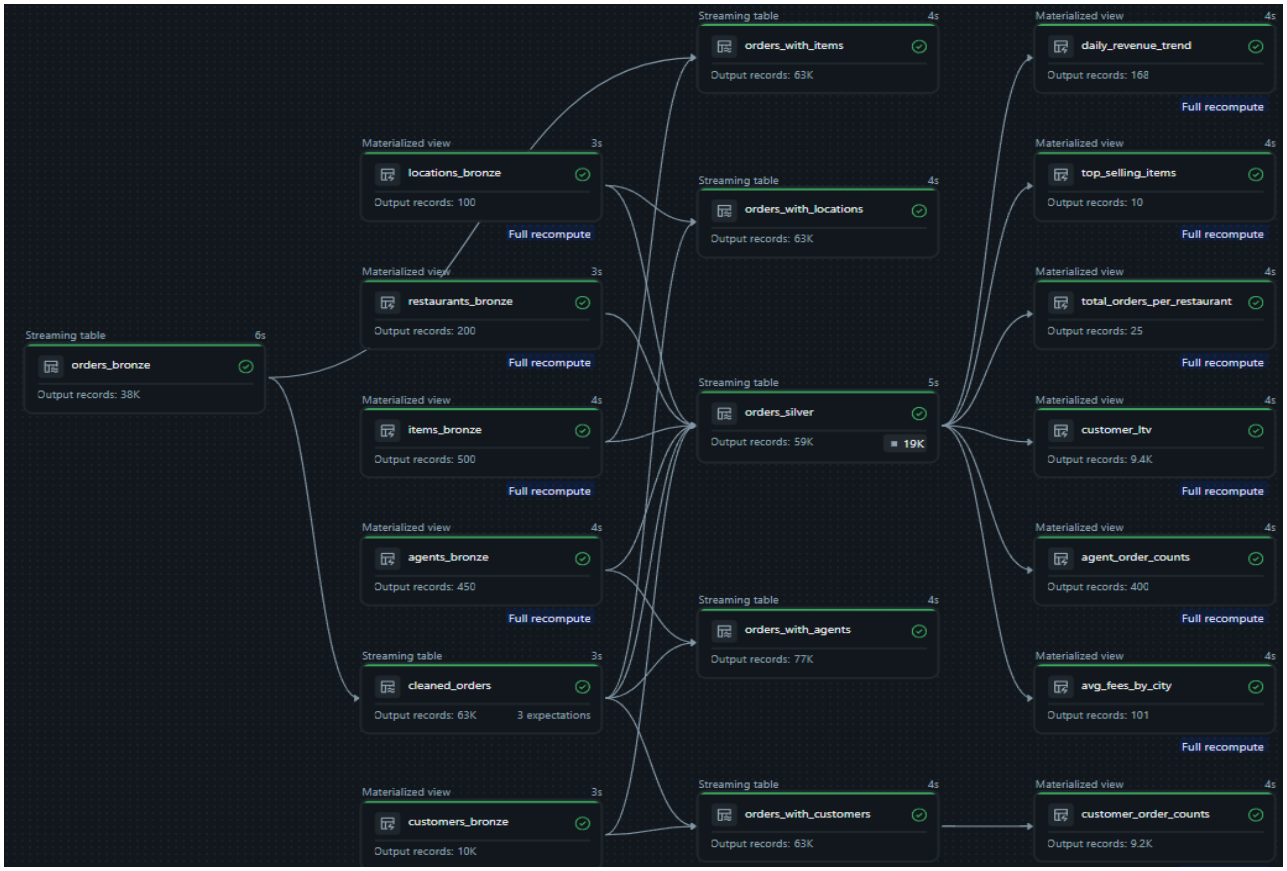
Gold Layer — **customer_order_counts**

Calculate total orders for each customer. Grouped orders_with_customers by customer_name and counted the orders. **customer_order_counts** defined.

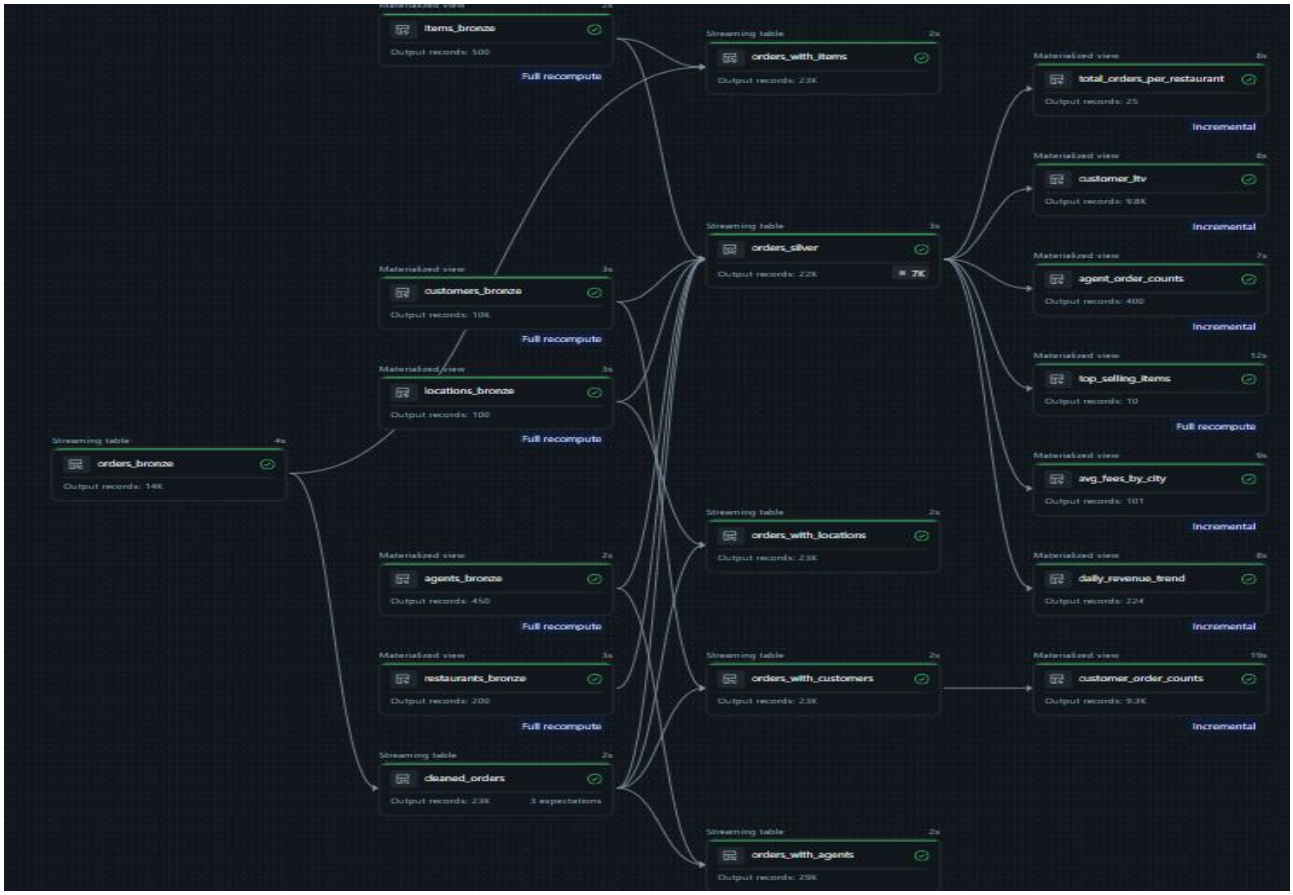
```
CREATE LIVE TABLE customer_order_counts AS
SELECT customer_name,COUNT(*) AS total_orders
FROM orders_with_customers
GROUP BY customer_name;
```

DLT Pipeline Execution

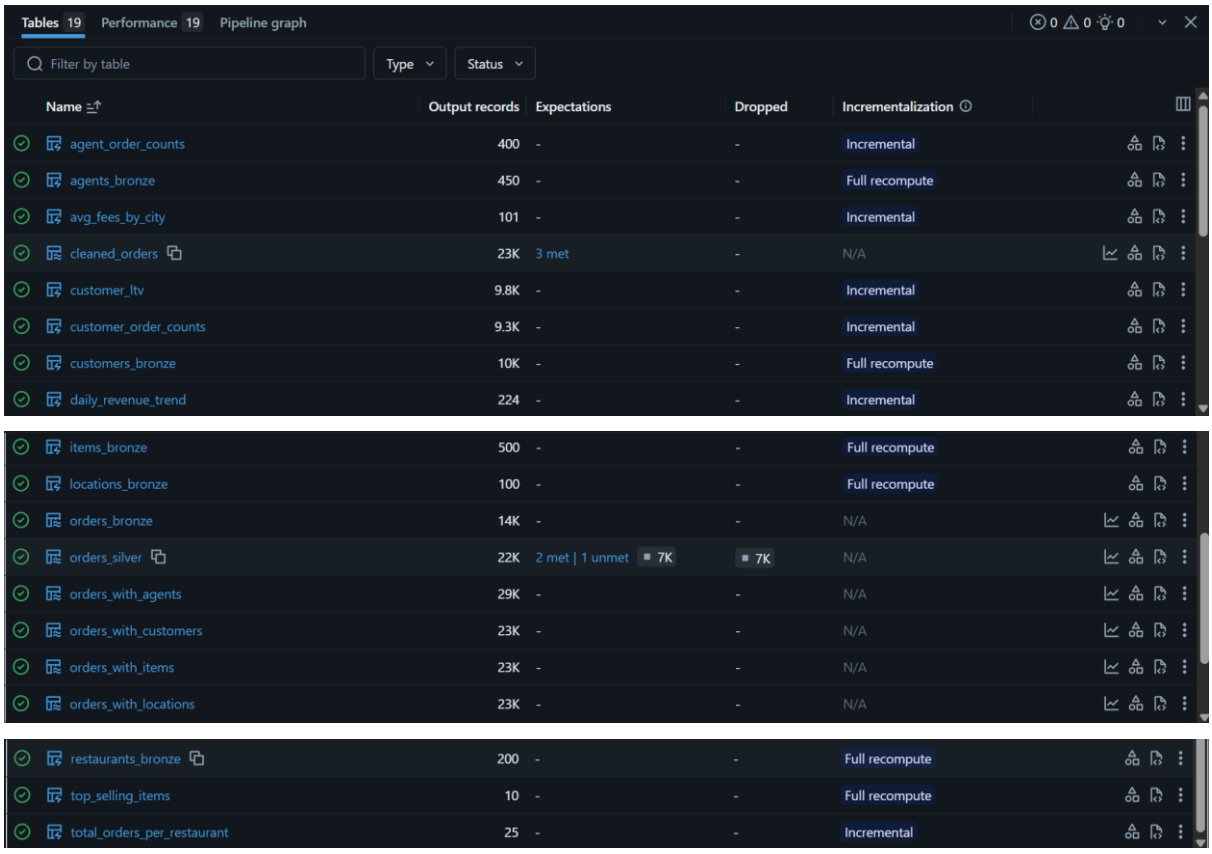
Execute the complete DineDash transformation pipeline. Ran the configured Lakeflow/DLT pipeline containing Bronze, Silver and Gold transformations. Pipeline execution completed and transformation dependencies were evaluated.

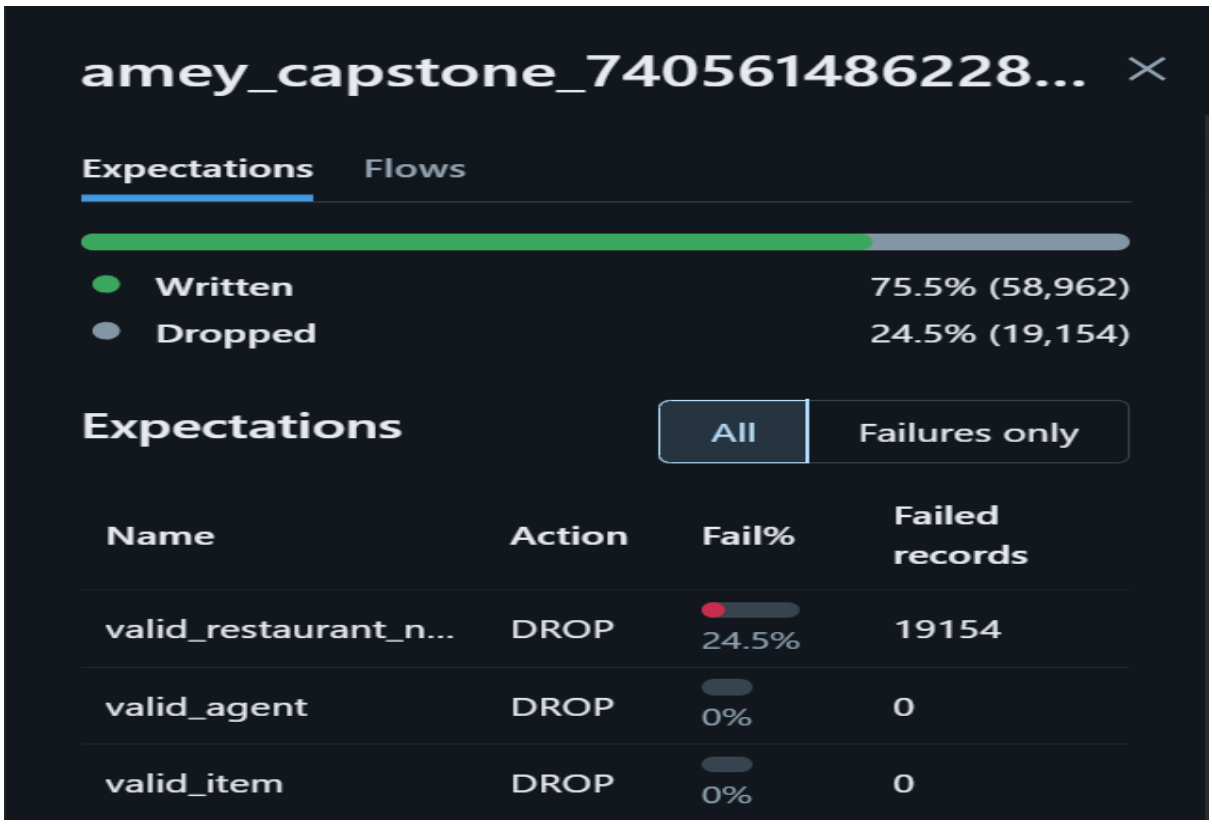


July and Aug Added



Data Quality Observation – July & August Incremental Load





During the July–August incremental pipeline run, the **orders_silver** data-quality expectations were evaluated. A total of 19,154 records (24.5%) were dropped, while 58,962 records (75.5%) were written. The dropped records failed the valid_restaurant_n... expectation, whereas the valid_agent and valid_item expectations had zero failed records.

Remaining 2024 Order Files Added to Streaming Source



Data Quality Validation — September to December

The September–December incremental pipeline run was completed and the resulting data-quality checks were reviewed for the remaining 2024 order files.

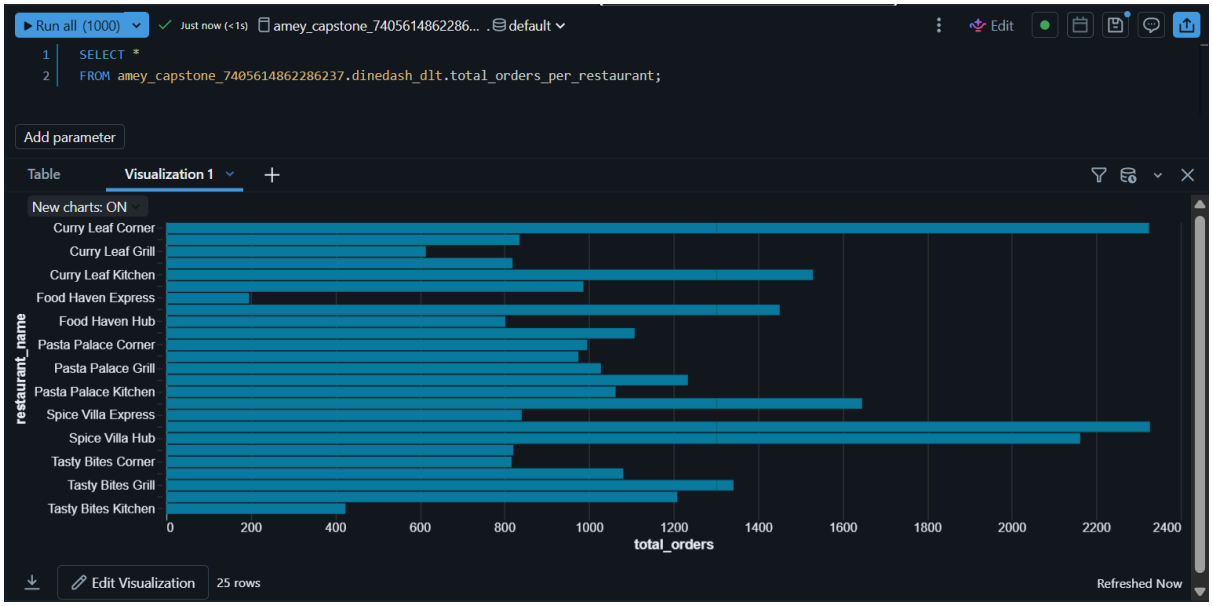
Name	Action	Fail%	Failed records
valid_restaurant_n...	DROP	24.9%	12012
valid_agent	DROP	0%	0
valid_item	DROP	0%	0

Dashboarding

To demonstrate incremental dashboard and workflow updates, the July–December order files were temporarily removed from the active source directory before the initial dashboard run and then reintroduced in stages for incremental processing.

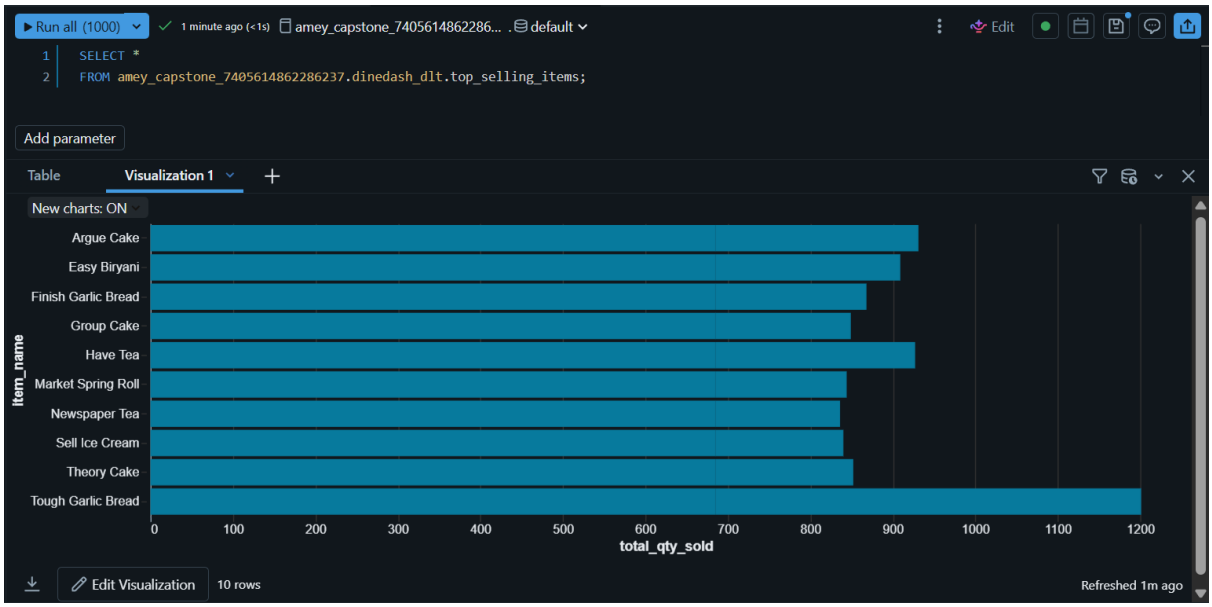
Dashboard Query — Total Orders per Restaurant

Prepare the restaurant order-volume metric for the dashboard. Queried the Gold **total_orders_per_restaurant** dataset and created a visualization. Restaurant-wise order counts are available for dashboard display.



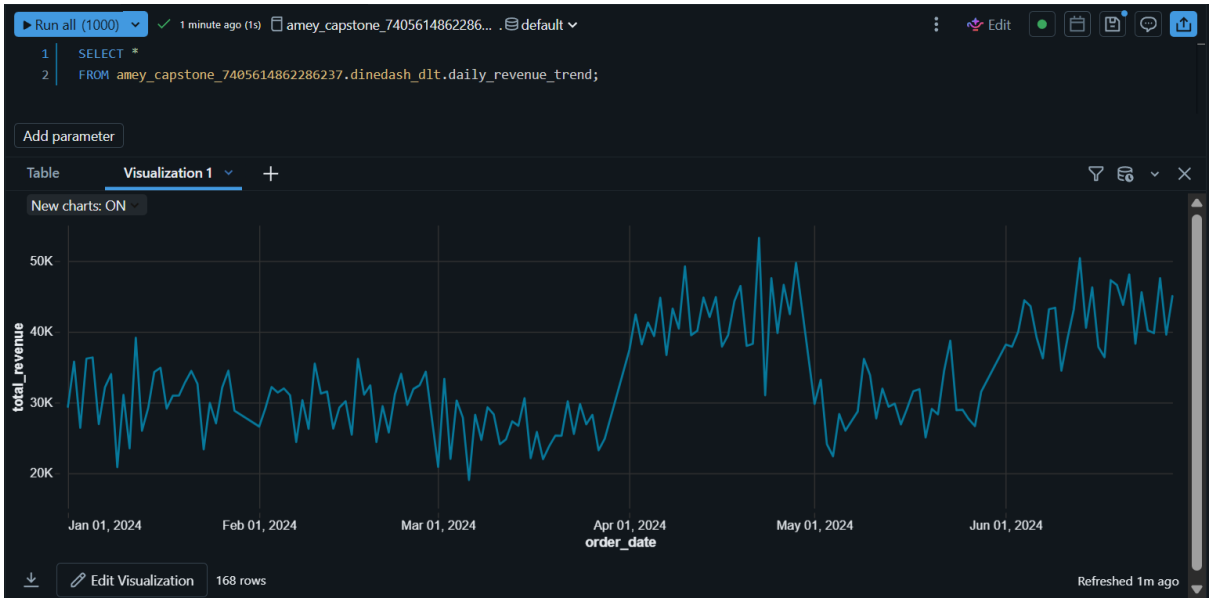
Dashboard Query — Top-Selling Items

Prepare the top-selling menu items metric for the dashboard. Queried the Gold **top_selling_items** dataset and created a visualization. The 10 top-selling items are available for dashboard display.



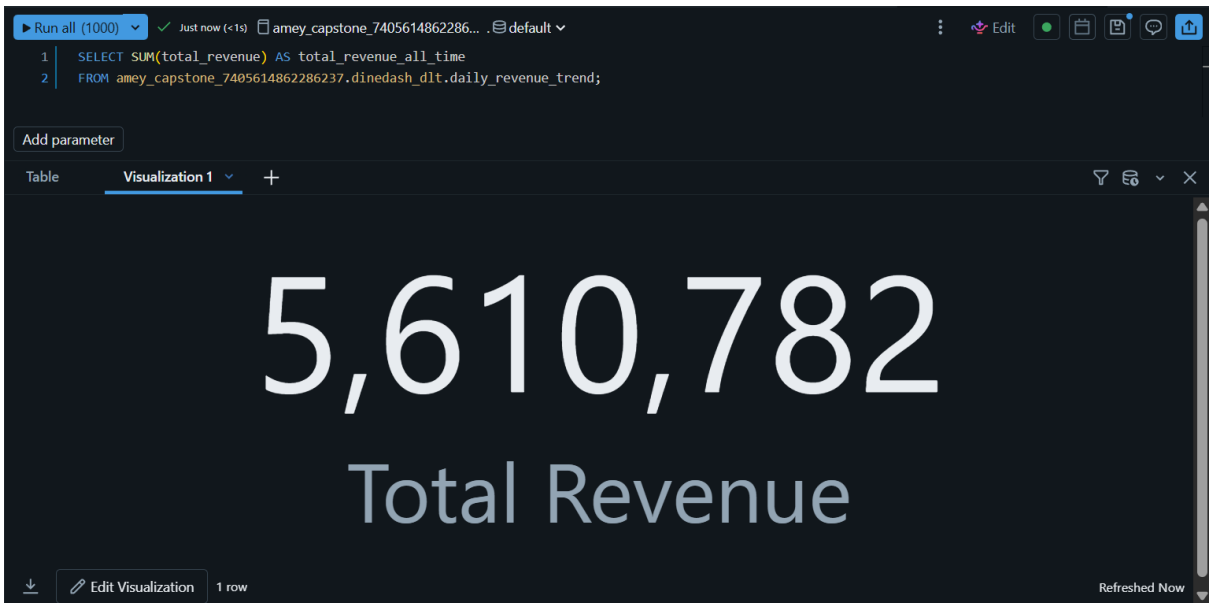
Dashboard Query — Daily Revenue Trend

Prepare daily revenue data for dashboard visualization. Queried the Gold **daily_revenue_trend** dataset and created a time-series visualization. Daily revenue values are available for dashboard display.



Dashboard Query — Total Revenue

Calculate the overall revenue generated across the available daily revenue data. Summed total_revenue from the Gold **daily_revenue_trend** dataset. A single total-revenue value was produced for the dashboard.



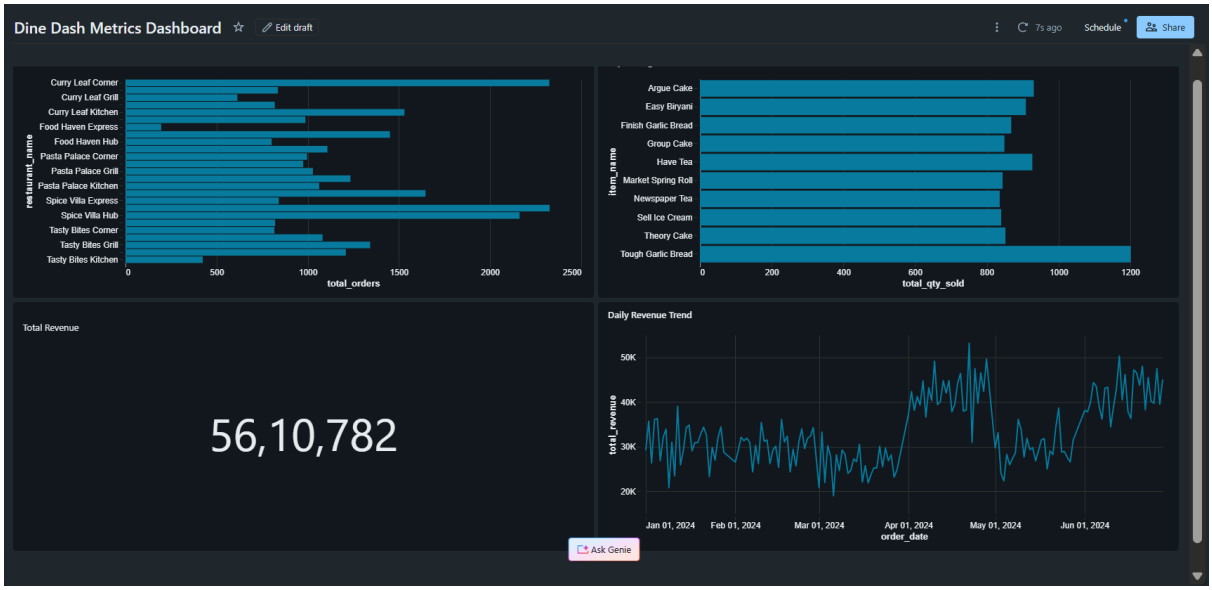
Dine Dash Metrics Dashboard

Create the dashboard that will contain the required business insights. Added the Total Orders Per Restaurant visualization to a new dashboard named Dine Dash Metrics Dashboard. The dashboard was created and is ready for the remaining visualizations.



Publish Dine Dash Metrics Dashboard

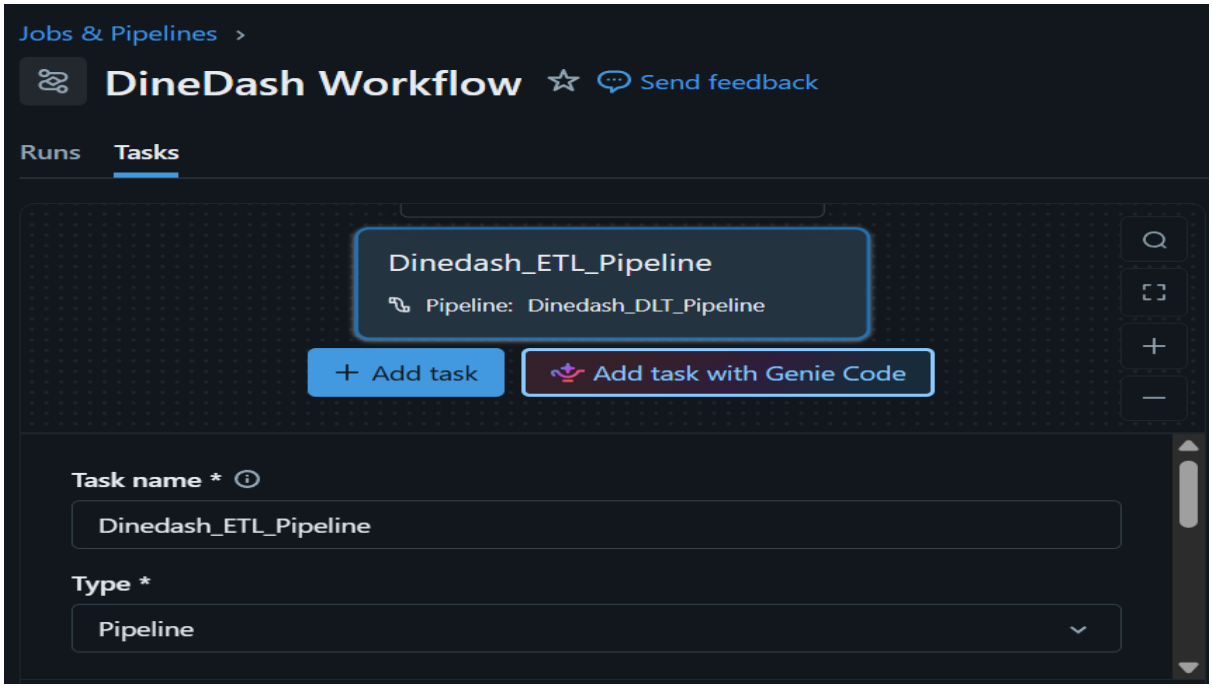
Publish the completed dashboard. Published Dine Dash Metrics Dashboard using the default publishing options. The dashboard is published and contains the required visualizations.



Workflow Incremental Update — July and August

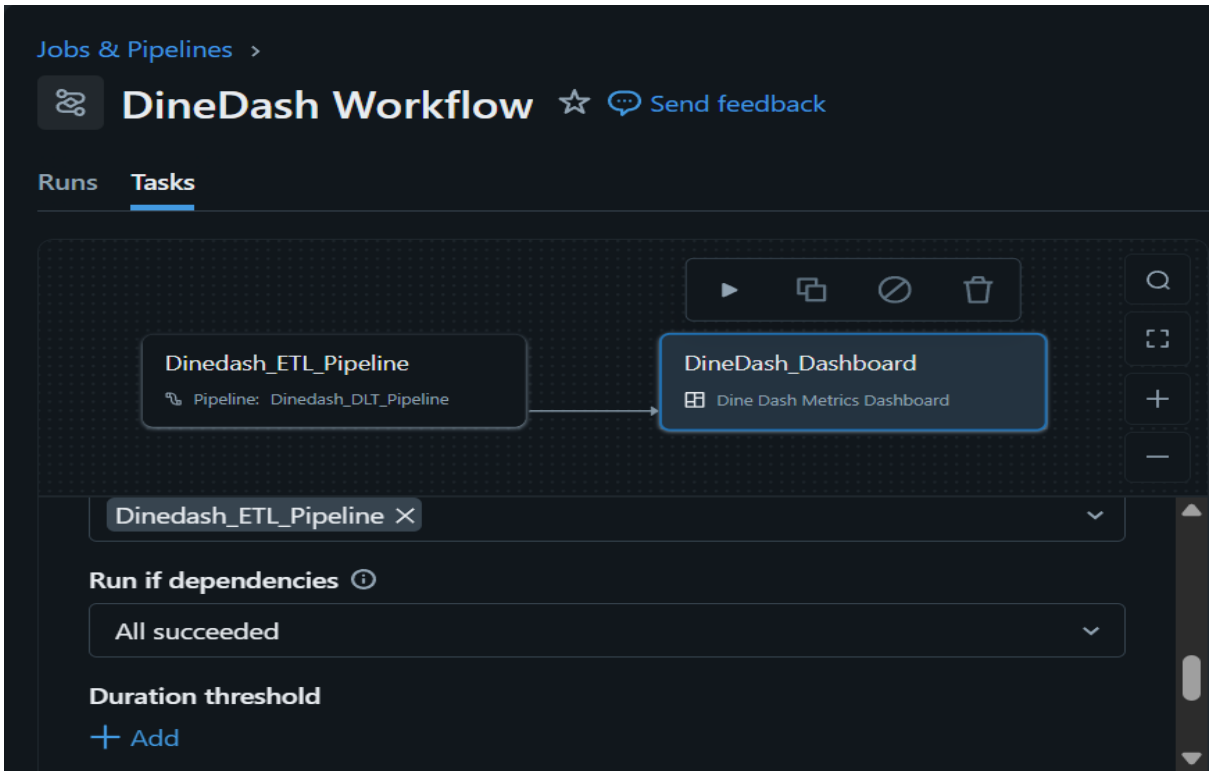
DineDash Workflow — Pipeline Task

Create the workflow required to execute the DineDash DLT pipeline. Created DineDash Workflow and added Dinedash_ETL_Pipeline as the first Pipeline task. The workflow contains the DLT pipeline as Task 1.



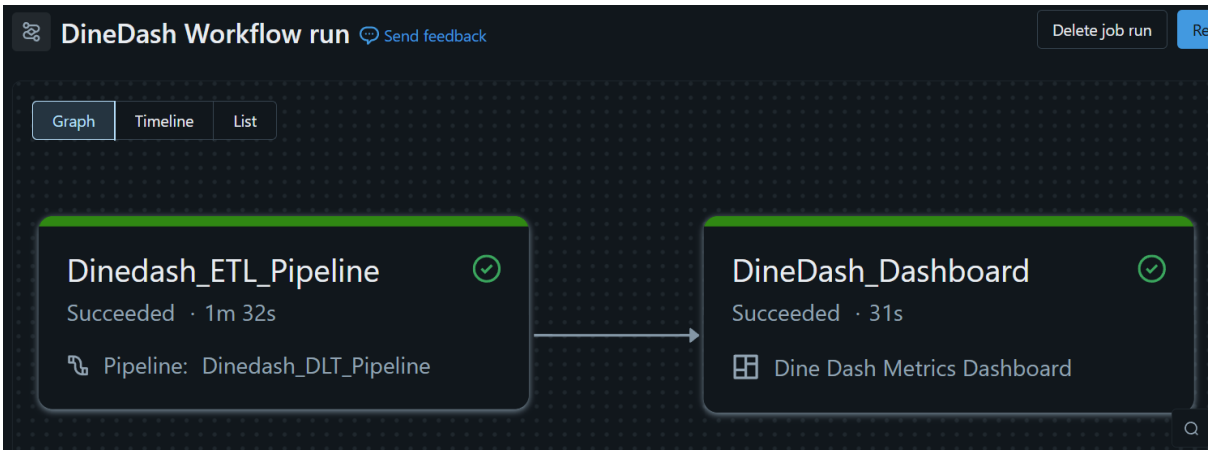
DineDash Workflow — Dashboard Task

Add the dashboard as the second workflow task. Added DineDash_Dashboard and configured it to run after Dinedash_ETL_Pipeline succeeds. The workflow now contains the required Pipeline → Dashboard dependency.



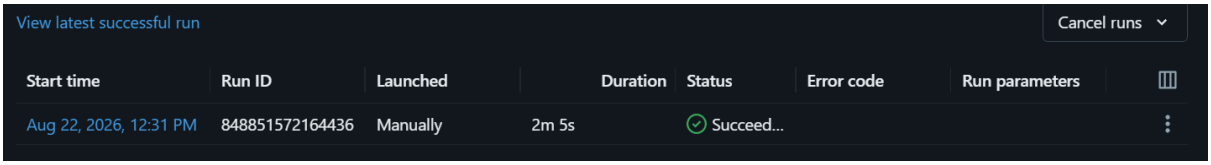
DineDash Workflow Execution

Execute the complete workflow. Ran the workflow using Run now, executing the DLT pipeline followed by the dashboard task. Both workflow tasks completed successfully.



Workflow Run Validation

Verify successful execution of both workflow tasks. Opened the latest workflow run and inspected the task execution details. The pipeline and dashboard tasks completed successfully.



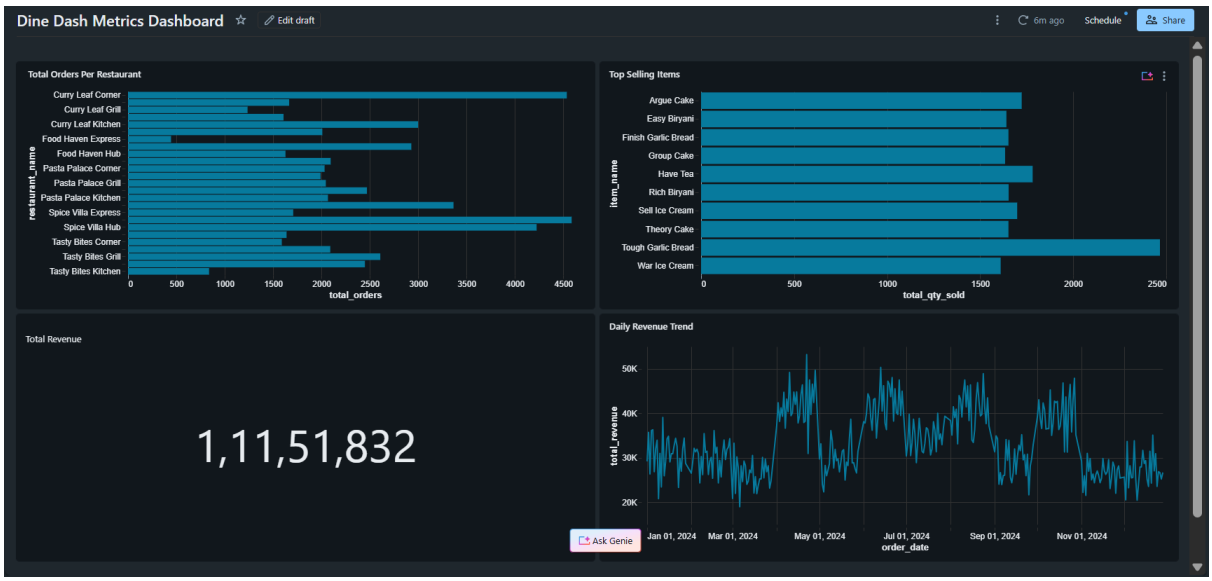
Dashboard Validation through Workflow

Verify that the workflow successfully refreshes the dashboard. Opened the dashboard link from the successful dashboard task and inspected the displayed metrics. The DineDash dashboard was refreshed and displayed the required business metrics.



Workflow Incremental Update — Remaining 2024 Files

The remaining order files from September to December were added to the existing orders folder. The workflow was then executed using Run Now, triggering the DLT pipeline and dashboard tasks to process the newly added data. The workflow successfully processed the remaining order data incrementally, and the dashboard was updated with the additional data.



Additional Analysis and Observations

Data quality observation: some order records reference restaurants for which the enriched restaurant name is unavailable. This is also reflected in the DLT data-quality results, where records failing the restaurant-name expectation are dropped. The same issue therefore appears both in analysis and in automated pipeline validation.

Some records contain a total order amount of $0, including records marked as delivered. This should be treated as a source-data quality issue rather than automatically as a meaningful business pattern.

Some customer signup dates appear substantially older than the operating history described in the case study. This is best interpreted as a characteristic of the supplied sample dataset rather than as an operational conclusion about DineDash.

The orders_bronze table intentionally preserves the raw order stream without the same dimension-level filtering applied to the five reference datasets. As a result, source-data issues can propagate into downstream analysis until Silver-layer expectations are applied.

The cuisine-diversity metric should be interpreted carefully because the supplied dataset contains a limited number of cuisine categories. A high diversity count therefore reflects the available category range as well as agent activity.

Conclusion

This capstone implemented the DineDash data-engineering workflow on Azure Databricks from source ingestion through SQL analysis, automated DLT processing, Gold-layer business metrics, dashboarding, and workflow orchestration. The project demonstrated manual incremental ingestion with COPY INTO as well as streaming and incremental processing with the DLT pipeline. The final workflow connected pipeline execution with dashboard refresh, while the data-quality checks exposed issues present in the supplied source data and showed how Silver-layer expectations can identify and handle those records.

Final Results

Source data was organized in a Unity Catalog Volume with separate dimension and streaming-order directories.

Dimension datasets were explored through direct reads, represented as temporary views, stored in manually defined Bronze Delta tables, and filtered into cleaned dimension tables.

orders_bronze was populated incrementally for January, February, and March using COPY INTO with record-count verification.

The required 12 Order Data Analysis and 15 Orders and Dimension Dataset Analysis questions were completed.

The DLT pipeline processed the initial January-June data, then processed July-August incrementally, followed by the remaining 2024 files.

DLT data-quality expectations were reviewed during incremental processing, including observation of dropped records and their failing expectation.

The required Gold-layer metrics were used to create and publish the DineDash dashboard.

The Databricks workflow executed the DLT pipeline and dashboard tasks and was rerun with additional source data to demonstrate incremental dashboard updates.